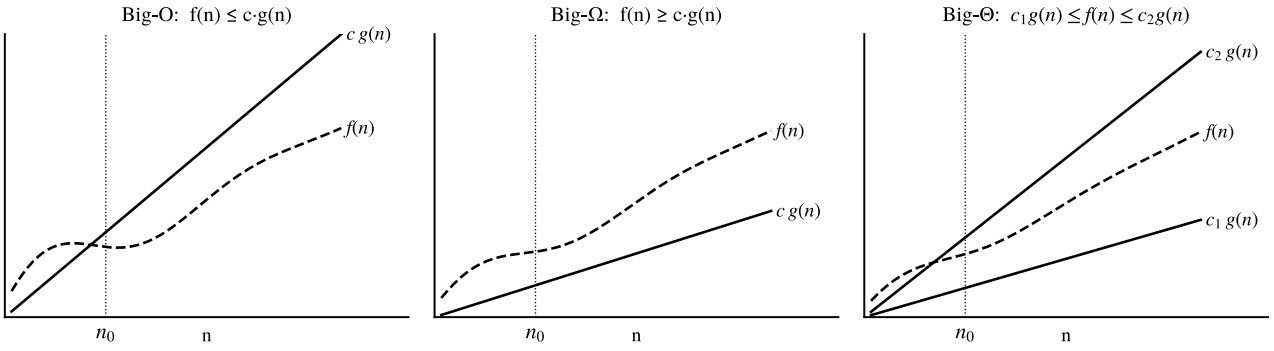
Design and Analysis of Algorithms (BCS-503)

Unit 1: Complexity, Recurrences & Sorting

Q1. Short Definitions (2 Marks Each)

(a) Define Big-O, Big-Omega and Big-Theta asymptotic notations.

- **Big-O (O) - Upper bound (worst case):** `f(n) = O(g(n))` if there exist positive constants **c** and **n0** such that **0 ≤ f(n) ≤ c·g(n) for all n ≥ n0**. Example: 3n + 2 = O(n), with c = 4 and n0 = 2.
- **Big-Omega (Ω) - Lower bound (best case):** `f(n) = Ω(g(n))` if there exist positive constants **c** and **n0** such that **0 ≤ c·g(n) ≤ f(n) for all n ≥ n0**. Example: 3n + 2 = Ω(n), with c = 3.
- **Big-Theta (Θ) - Tight bound:** `f(n) = Θ(g(n))` if there exist positive constants **c1, c2, n0** such that **0 ≤ c1·g(n) ≤ f(n) ≤ c2·g(n) for all n ≥ n0**. Example: 3n + 2 = Θ(n). Also, f(n) = Θ(g(n)) if and only if f(n) = O(g(n)) and f(n) = Ω(g(n)).



Asymptotic bounds: O gives an upper bound, Ω a lower bound and Θ a tight bound (for all $n \geq n_0$)

(b) Differentiate between stable and unstable sorting algorithms.

Stable Sorting	Unstable Sorting
Keeps the **relative order of equal keys** as in the input	May **change the relative order** of equal keys
Example: (5a, 3, 5b) gives (3, 5a, 5b)	Example: (5a, 3, 5b) may give (3, 5b, 5a)
Algorithms: Bubble, Insertion, Merge, Counting, Radix sort	**Algorithms:** Quick, Heap, Selection, Shell sort
Needed for multi-key sorting (sort by name, then by marks)	Fine when equal keys need not keep their order

---

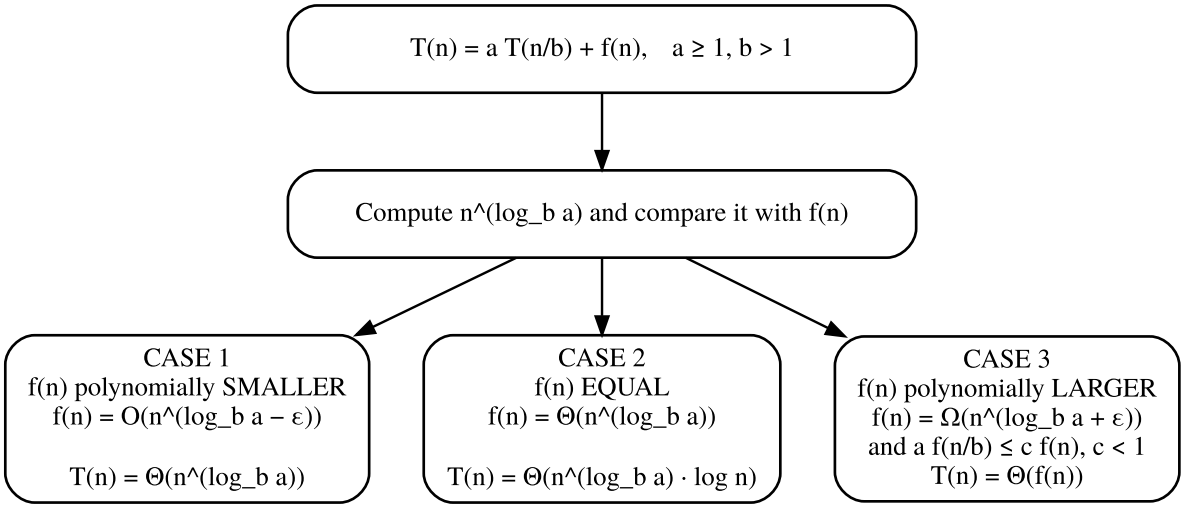
Q2. Solve recurrence relations using the Master Theorem or Recursion Tree method (e.g., T(n) = 2T(n/2) + n and T(n) = 3T(n/9) + n). (7 Marks)

1. Introduction / Definition

A **recurrence relation** expresses the running time T(n) of a recursive algorithm in terms of its running time on **smaller inputs**. The **Master Theorem** gives a direct solution for **divide-and-conquer recurrences** of the form `T(n) = a·T(n/b) + f(n)`, where a ≥ 1 and b > 1. The **Recursion Tree method** draws the recursive calls as a tree

and **adds the cost of every level**.

2. Flowchart - Master Theorem Decision



Master Theorem: compare f(n) with n^(log_b a)

3. Core Explanation / Working

Master Theorem (statement): for `T(n) = a·T(n/b) + f(n)`:

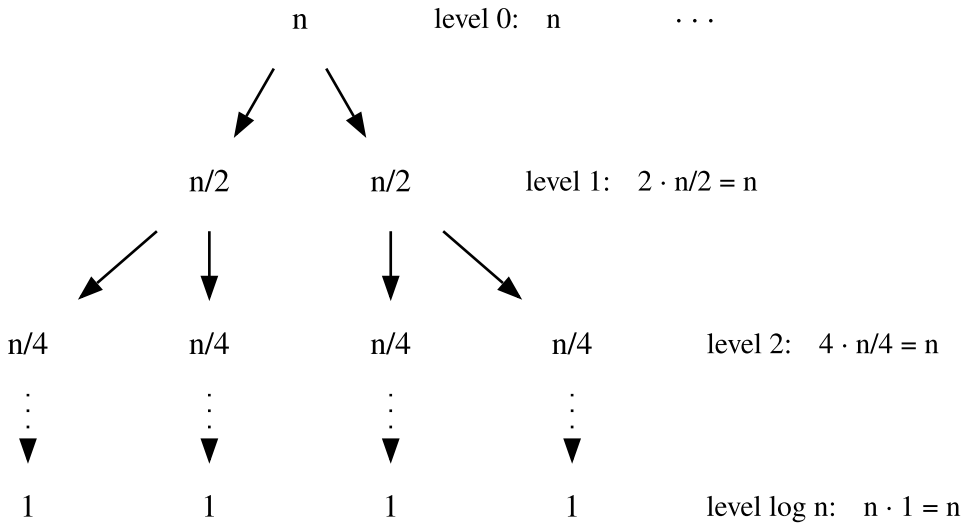
- **Case 1:** if f(n) = O(n^(log_b a − ε)) for some ε > 0, then **T(n) = Θ(n^(log_b a))**.
- **Case 2:** if f(n) = Θ(n^(log_b a)), then **T(n) = Θ(n^(log_b a) · log n)**.
- **Case 3:** if f(n) = Ω(n^(log_b a + ε)) for some ε > 0, **and** a·f(n/b) ≤ c·f(n) for some c < 1 (the regularity condition), then **T(n) = Θ(f(n))**.

Example 1: `T(n) = 2T(n/2) + n`

(i) Master Theorem

- a = 2, b = 2, f(n) = n.
- n^(log_b a) = n^(log_2 2) = n^1 = **n**.
- f(n) = n = Θ(n^(log_b a)), so this is **Case 2**.
- **T(n) = Θ(n log n)** (the Merge Sort recurrence).

(ii) Recursion Tree



Recursion tree for T(n) = 2T(n/2) + n: every level costs n, and there are $log_2 n + 1$ levels

$$T(n) = \underbrace{n + n + \cdots + n}_{\log_2 n \,+\, 1 \text{ levels}} = n\left(\log_2 n + 1\right) = \Theta\left(n \log n\right)$$

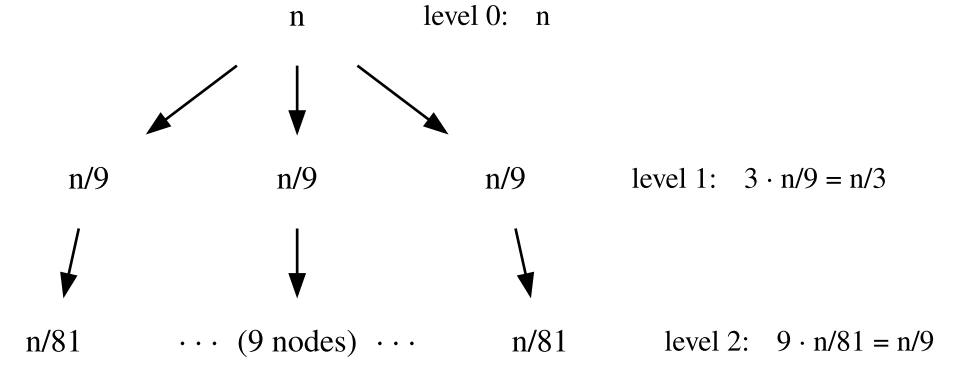
- At level i there are 2^i nodes, each costing n/2^i, so **each level costs n**.
- The recursion stops when n/2^i = 1, that is i = log2 n.
- **T(n) = n(log2 n + 1) = Θ(n log n)**

Example 2: `T(n) = 3T(n/9) + n`

(i) Master Theorem

- a = 3, b = 9, f(n) = n.
- n^(log_b a) = n^(log_9 3) = n^(1/2) = **√n** (since 9^(1/2) = 3).
- f(n) = n = Ω(n^(1/2 + ε)) with ε = 1/2, so f(n) is polynomially larger. Check Case 3.
- *Regularity:* a·f(n/b) = 3·(n/9) = n/3 ≤ c·n with **c = 1/3 < 1**. Satisfied.
- **T(n) = Θ(f(n)) = Θ(n)**

(ii) Recursion Tree (verification)



Recursion tree for T(n) = 3T(n/9) + n: level costs shrink by a factor of 3

$$T(n) \leq n\left(1 + \frac{1}{3} + \frac{1}{9} + \cdots\right) = n \cdot \frac{1}{1 - \frac{1}{3}} = \frac{3n}{2} = \Theta(n)$$

- Level costs form a **decreasing geometric series** (ratio 1/3), so the **root dominates**.
- **T(n) = Θ(n)**

Example 3 (Case 1, extra practice): `T(n) = 9T(n/3) + n`

- n^(log_3 9) = n². f(n) = n = O(n^(2 − ε)) with ε = 1, so this is **Case 1**.
- **T(n) = Θ(n²)**

4. Key Characteristics - Methods of Solving Recurrences

Method	Idea	Best For	Limitation
Substitution	Guess the answer, prove by induction	Any recurrence	Needs a good guess
Recursion Tree	Sum the cost level by level	Visualising, making a guess	Can be lengthy
Master Theorem	Compare f(n) with n^(log_b a)	`aT(n/b) + f(n)` form	Gaps between cases; not for T(n−1) forms
Iteration	Expand repeatedly	Simple linear recurrences	Algebra can be heavy

5. Advantages & Disadvantages

Advantages

- The Master Theorem gives a **quick, direct answer** for divide-and-conquer algorithms.
- The Recursion Tree gives an **intuitive picture** of where the work is done.
- Both help in **comparing algorithms** (Merge Sort Θ(n log n) vs Bubble Sort Θ(n²)).

Disadvantages

- The Master Theorem **does not apply** when f(n) falls in the gaps (for example `T(n) = 2T(n/2) + n/log n`) or to **subtract-type** recurrences like T(n) = T(n−1) + n.
- Recursion trees for **unbalanced** splits are tedious to sum.

6. Conclusion / Application

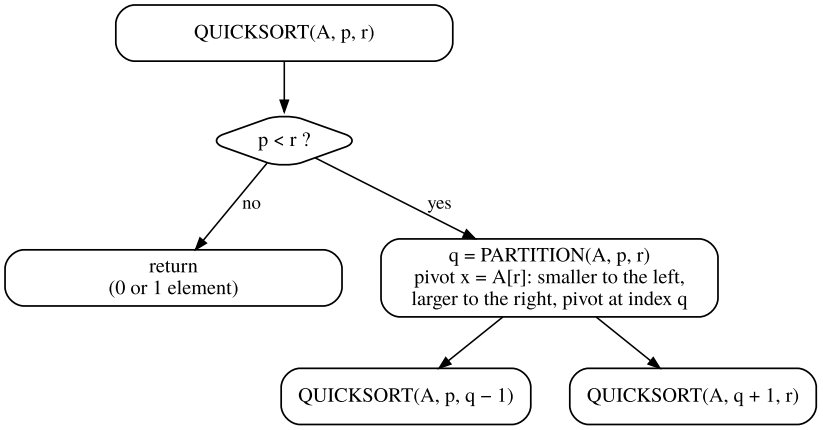
These methods are used to analyse **Merge Sort, Binary Search, Strassen's matrix multiplication and Karatsuba multiplication**.

Q3. Explain the Quick Sort algorithm, trace it on a given array, and justify why it is often preferred over Merge Sort. (7 Marks)

1. Introduction / Definition

Quick Sort is a **divide-and-conquer, in-place** sorting algorithm developed by **C.A.R. Hoare**. It picks a **pivot** element and **partitions** the array so that all smaller elements are on its left and all larger elements are on its right. It then sorts the two parts **recursively**. Its average time is **O(n log n)**.

2. Flowchart - Quick Sort



Quick Sort

3. Core Explanation / Working

Algorithm (Lomuto partition, CLRS)

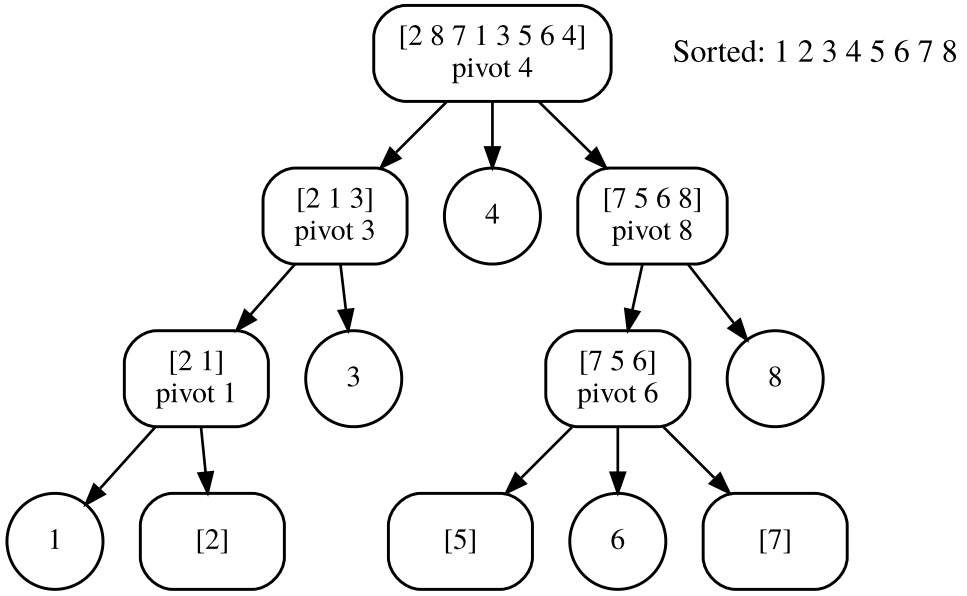
```
QUICKSORT(A, p, r)
1. if p < r
2.     q = PARTITION(A, p, r)
3.     QUICKSORT(A, p, q - 1)
4.     QUICKSORT(A, q + 1, r)

PARTITION(A, p, r)
1. x = A[r]                 // pivot = last element
2. i = p - 1
3. for j = p to r - 1
4.     if A[j] <= x
5.         i = i + 1
6.         swap A[i] and A[j]
7. swap A[i + 1] and A[r]
8. return i + 1
```

Trace on A = [2, 8, 7, 1, 3, 5, 6, 4] (indices 1 to 8, pivot = 4)

Step	j	A[j]	A[j] ≤ 4?	i	Array after step
Start	-	-	-	0	2 8 7 1 3 5 6 **4**
1	1	2	Yes, swap A1,A1	1	2 8 7 1 3 5 6 4
2	2	8	No	1	2 8 7 1 3 5 6 4
3	3	7	No	1	2 8 7 1 3 5 6 4
4	4	1	Yes, swap A2,A4	2	2 1 7 8 3 5 6 4
5	5	3	Yes, swap A3,A5	3	2 1 3 8 7 5 6 4
6	6	5	No	3	2 1 3 8 7 5 6 4
7	7	6	No	3	2 1 3 8 7 5 6 4
End	-	-	swap A4, A8	-	2 1 3 **[4]** 7 5 6 8

Recursive calls



Recursive calls of Quick Sort on [2, 8, 7, 1, 3, 5, 6, 4] (circled = pivot placed in its final position)

Complexity

- **Best / average case:** the pivot splits the array roughly in half: T(n) = 2T(n/2) + n = **Θ(n log n)**.
- **Worst case:** the pivot is always the smallest or largest element (an already-sorted array): T(n) = T(n−1) + n = **Θ(n²)**.
- **Space:** O(log n) for the recursion stack on average. The sort is **in-place**.
- **Avoiding the worst case: randomized pivot** or **median-of-three** pivot selection.

Why Quick Sort is often preferred over Merge Sort

1. **In-place:** it needs only O(log n) extra stack space, while Merge Sort needs an **O(n) auxiliary array**.
2. **Better cache performance:** it works on contiguous memory, so it has good **locality of reference**.
3. **Smaller constant factors:** the inner loop is very simple (one comparison, occasional swap), so it is faster in practice.
4. **The worst case is rare:** with a randomized pivot, the expected time is O(n log n) for every input.
5. It is used in standard libraries (C `qsort`, and Java's `Arrays.sort()` for primitive types uses dual-pivot Quick Sort).

4. Key Characteristics - Quick Sort vs. Merge Sort

Basis	Quick Sort	Merge Sort
Strategy	Partition, then recurse (work done **before** recursion)	Recurse, then merge (work done **after** recursion)
Best / average time	O(n log n)	O(n log n)
Worst time	**O(n²)**	**O(n log n)**
Extra space	**O(log n)**, in-place	O(n)
Stable	No	**Yes**
Cache performance	**Excellent**	Moderate
Best for	Arrays in RAM	Linked lists, external sorting

5. Advantages & Disadvantages

Advantages

- **Fastest in practice** for large in-memory arrays.
- **In-place**: very little extra memory.
- Average case is **O(n log n)**, and randomization makes the worst case unlikely.

Disadvantages

- **Worst case O(n^2)** on sorted or reverse-sorted input when the pivot is chosen badly.
- **Not stable**: the order of equal elements may change.
- Its recursive nature can cause **stack overflow** on bad inputs.

6. Conclusion / Application

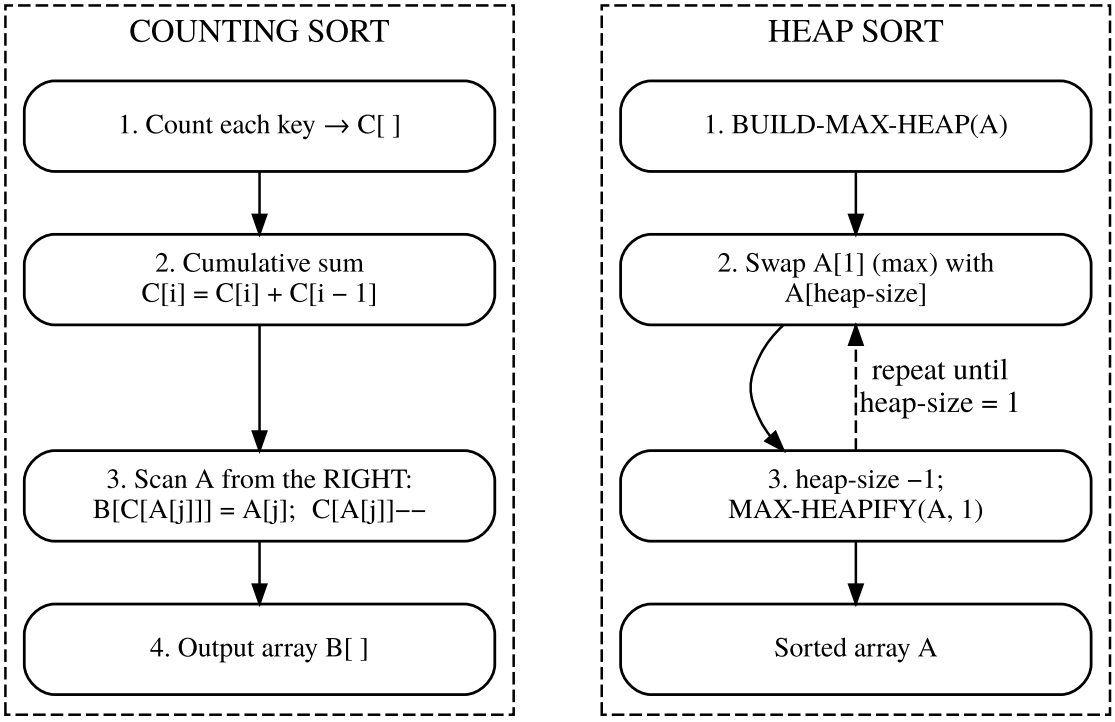
Quick Sort is the basis of **C's `qsort()`, C++ `std::sort` (introsort) and Java's primitive-array sort**, and it is used widely in database query processing.

Q4. Trace Counting Sort and Heap Sort on a given array of numbers. (7 Marks)

1. Introduction / Definition

Counting Sort is a **non-comparison, stable** sorting algorithm for integers in the range 0 to k. It **counts the occurrences** of each value and uses cumulative counts to place elements, in **O(n + k)** time. **Heap Sort** is a **comparison-based, in-place** algorithm. It builds a **max-heap** and then repeatedly moves the maximum to the end, in **O(n log n)** time.

2. Block Diagram



Steps of Counting Sort and Heap Sort

3. Core Explanation / Working

(A) Counting Sort trace: `A = [2, 5, 3, 0, 2, 3, 0, 3]`, n = 8, k = 5

Step 1 - Count the occurrences, C[0..5]

Index i	0	1	2	3	4	5
C[i] (count)	2	0	2	3	0	1

Step 2 - Cumulative count (C[i] = number of elements ≤ i)

Index i	0	1	2	3	4	5
C[i] (cumulative)	2	2	4	7	7	8

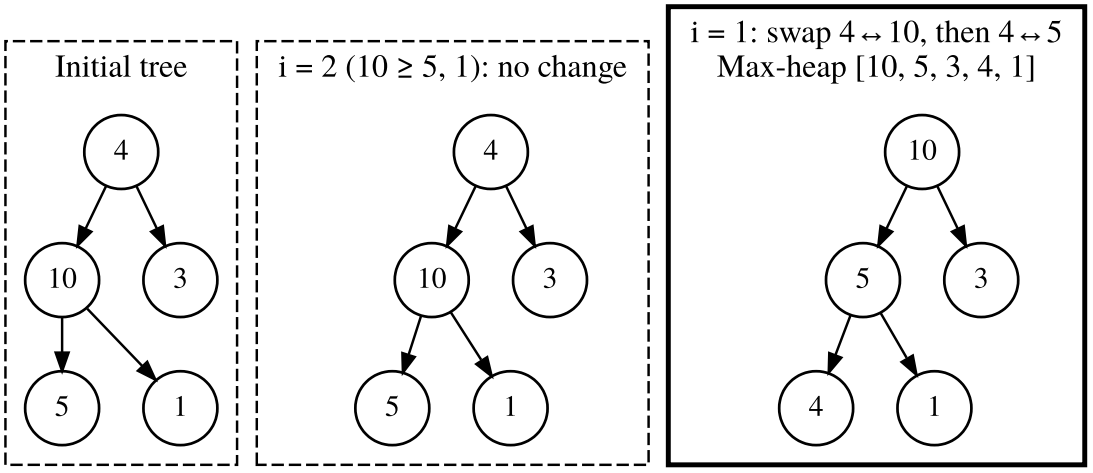
Step 3 - Place elements, scanning A from right to left (this keeps the sort stable)

j	A[j]	C[A[j]] → position in B	B after placing (positions 1-8)	C[A[j]] after decrement
8	3	7	_ _ _ _ _ _ 3 _	C[3] = 6
7	0	2	_ 0 _ _ _ _ 3 _	C[0] = 1
6	3	6	_ 0 _ _ _ 3 3 _	C[3] = 5
5	2	4	_ 0 _ 2 _ 3 3 _	C[2] = 3
4	0	1	0 0 _ 2 _ 3 3 _	C[0] = 0
3	3	5	0 0 _ 2 3 3 3 _	C[3] = 4
2	5	8	0 0 _ 2 3 3 3 5	C[5] = 7
1	2	3	0 0 2 2 3 3 3 5	C[2] = 2

Sorted output: B = [0, 0, 2, 2, 3, 3, 3, 5]. Time **O(n + k)**, space **O(n + k)**.

(B) Heap Sort trace: `A = [4, 10, 3, 5, 1]`

Step 1 - BUILD-MAX-HEAP (heapify from i = ⌊n/2⌋ = 2 down to 1)



BUILD-MAX-HEAP on [4, 10, 3, 5, 1]

Step 2 - Repeatedly extract the maximum

Pass	Swap root with last	After swap	Heapify result (heap part / sorted part)
1	10 ↔ 1	[1, 5, 3, 4 / 10]	1 ↔ 5, then 1 ↔ 4: **[5, 4, 3, 1 / 10]**
2	5 ↔ 1	[1, 4, 3 / 5, 10]	1 ↔ 4: **[4, 1, 3 / 5, 10]**
3	4 ↔ 3	[3, 1 / 4, 5, 10]	3 ≥ 1, no change: **[3, 1 / 4, 5, 10]**
4	3 ↔ 1	[1 / 3, 4, 5, 10]	Single element

Sorted output: A = [1, 3, 4, 5, 10]. Time **O(n log n)** in all cases, space **O(1)**.

4. Key Characteristics - Counting Sort vs. Heap Sort

Basis	Counting Sort	Heap Sort
Type	Non-comparison	Comparison-based
Time complexity	**O(n + k)**	**O(n log n)** (all cases)
Extra space	O(n + k)	**O(1)**, in-place
Stable	**Yes**	No
Input restriction	Integers in a small range 0..k	Any comparable keys
Data structure	Count array	Binary max-heap

5. Advantages & Disadvantages

Counting Sort

- *Advantages:* **linear time** when k = O(n); **stable**, so it is used inside Radix Sort.
- *Disadvantages:* **impractical for a large range** k (huge count array); works only for integers or discrete keys.

Heap Sort

- *Advantages:* a **guaranteed O(n log n)** worst case; **in-place**, with O(1) space.
- *Disadvantages:* **not stable**; **poor cache performance**, so it is slower than Quick Sort in practice.

6. Conclusion / Application

Counting Sort is used for **sorting exam marks or ages and inside Radix Sort**. Heap Sort and heaps are used in **priority queues, OS job scheduling and the introsort fallback** of C++ `std::sort`.

Unit 2: Advanced Data Structures

Q1. Short Definitions (2 Marks Each)

(a) Properties of a Red-Black Tree

A Red-Black Tree is a self-balancing binary search tree that satisfies these **five properties**:

1. Every node is either **RED or BLACK**.
2. The **root is always BLACK**.
3. Every leaf (**NIL / external node**) is **BLACK**.
4. If a node is **RED**, **both its children are BLACK**. There are no two consecutive red nodes on any path.
5. For every node, **all simple paths from that node down to its descendant leaves contain the same number of BLACK nodes** (the **black-height**).

- *Consequence:* the height is at most **2 log2(n + 1)**, so search, insert and delete take **O(log n)** time.

(b) Define a B-Tree of order m

A **B-Tree of order m** is a self-balancing **m-way search tree** in which:

1. Every node has **at most m children** and **at most m – 1 keys**.

2. Every internal node except the root has **at least ⌈m/2⌉ children**, that is at least ⌈m/2⌉ − 1 keys.
3. The root has **at least 2 children** if it is not a leaf.
4. **All leaves are at the same level.**
5. The keys in each node are in **sorted order**, and a node with k keys has **k + 1 children**.

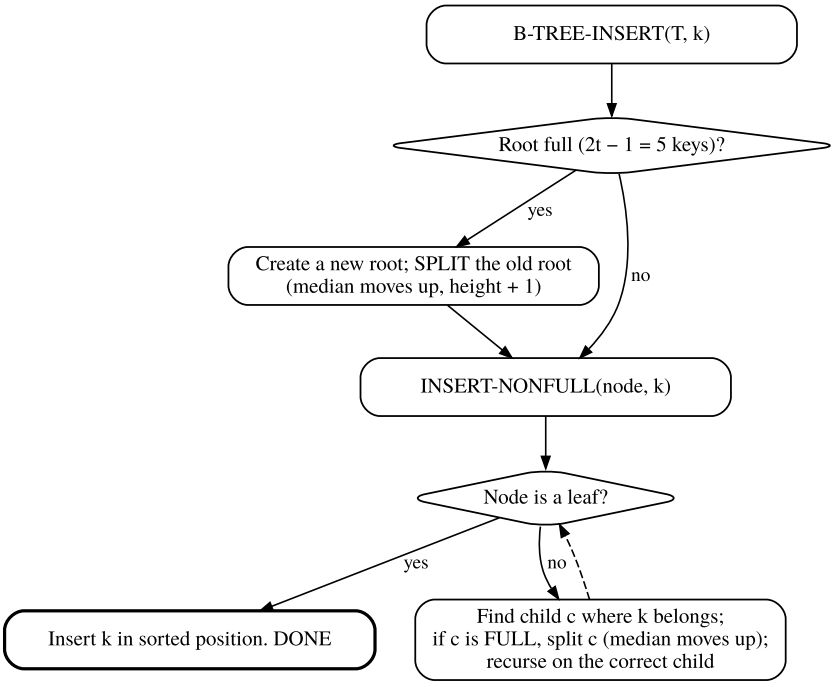
- (In CLRS notation with **minimum degree t**, each non-root node has between **t − 1 and 2t − 1 keys**.)

Q2. Perform step-by-step insertions into an empty B-Tree of a given degree (e.g., minimum degree t = 3). (7 Marks)

1. Introduction / Definition

A **B-Tree of minimum degree t** is a balanced multi-way search tree in which every node except the root has **at least t − 1 and at most 2t − 1 keys**, and **all leaves have the same depth**. For **t = 3**, each node holds **2 to 5 keys** (the root holds 1 to 5) and has up to **6 children**. Insertion always happens at a **leaf**, and **full nodes are split** to keep the tree balanced.

2. Flowchart - B-Tree Insertion (CLRS, proactive splitting)

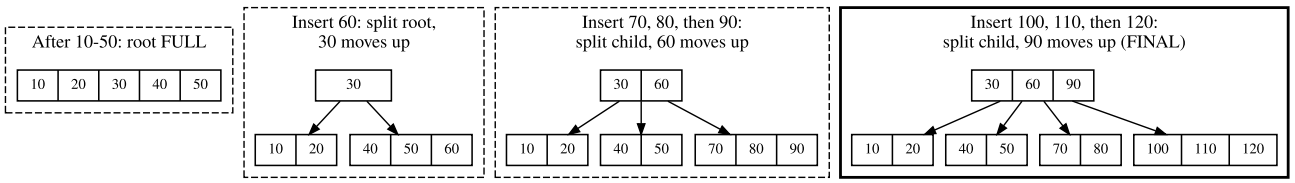


B-Tree insertion with proactive splitting (CLRS)

3. Core Explanation / Working

Split operation: a full node with 2t − 1 = 5 keys `[k1 k2 k3 k4 k5]` is split into `[k1 k2]` and `[k4 k5]`, and the **median k3 moves up** to the parent.

Example: insert **10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120** into an empty B-Tree with **t = 3** (max 5 keys per node).



B-Tree (t = 3, max 5 keys per node): inserting 10, 20, ..., 120

Step	Keys inserted	Action	Result
1-5	10, 20, 30, 40, 50	Fill the root	[10 | 20 | 30 | 40 | 50] (full)
6	60	Root full: **split**, 30 moves up	[30] → [10 | 20], [40 | 50 | 60]
7-8	70, 80	Into the right leaf	[40 | 50 | 60 | 70 | 80] (full)
9	90	Child full: **split**, 60 moves up	[30 | 60] → [10 | 20], [40 | 50], [70 | 80 | 90]
10-11	100, 110	Into the rightmost leaf	[70 | 80 | 90 | 100 | 110] (full)
12	120	Child full: **split**, 90 moves up	[30 | 60 | 90] → [10 | 20], [40 | 50], [70 | 80], [100 | 110 | 120]

Final tree check:

- The root has 3 keys and 4 children.
- Every leaf has 2 to 3 keys, which is ≥ t − 1 = 2 and ≤ 2t − 1 = 5.
- All leaves are at the **same depth (1)**.

Note: CLRS splits a full node **on the way down, before inserting** (proactive splitting), so a split never has to travel back up the tree.

4. Key Characteristics - Parameters for t = 3

Parameter	Formula	Value for t = 3
Max keys per node	2t − 1	5
Min keys (non-root)	t − 1	2
Max children	2t	6
Min children (non-root internal)	t	3
Max height for n keys	log_t ((n + 1)/2)	O(log n)
Insert / search time	O(t · log_t n)	O(log n)

5. Advantages & Disadvantages

Advantages

- **Always balanced**: all leaves at the same level guarantee **O(log n)** operations.
- **Few disk accesses**: one node equals one disk block, which is ideal for databases.
- Proactive splitting makes insertion a **single top-down pass**.

Disadvantages

- Nodes may be **only half full**, which wastes space.
- Split and merge logic is **complex** to implement.

6. Conclusion / Application

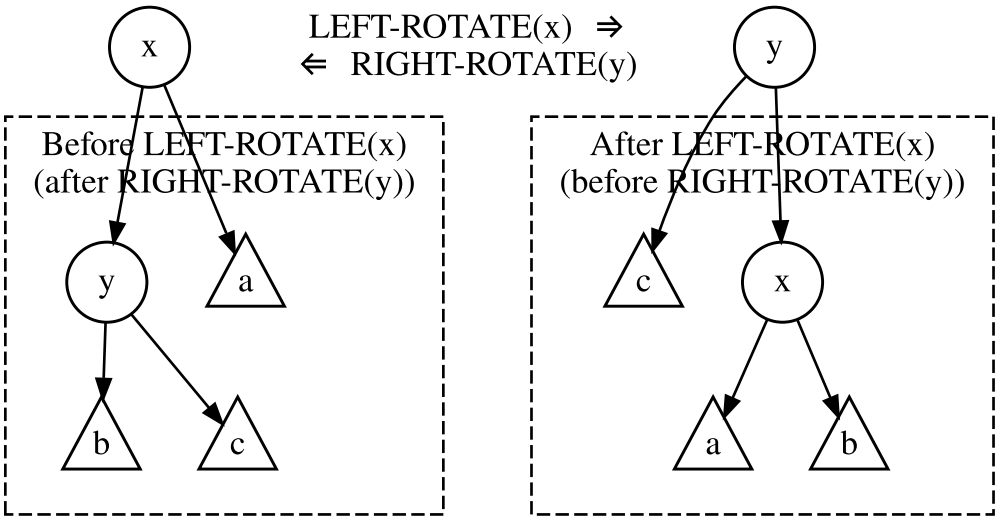
B-Trees are used for **database indexing (MySQL, Oracle) and file systems (NTFS, HFS+, ext4 directories)**, where data lives on disk.

Q3. Demonstrate key insertions and rotations in a Red-Black Tree. (7 Marks)

1. Introduction / Definition

A **Red-Black Tree** is a **self-balancing binary search tree** in which every node is coloured **red or black**. The colouring rules keep the height **O(log n)**. A new node is always inserted as **RED**. If this breaks a property (two reds in a row), the tree is fixed by **recolouring and rotations** (left or right).

2. Diagram - Rotations



Left and right rotation (in-order sequence a, x, b, y, c is unchanged; O(1) time)

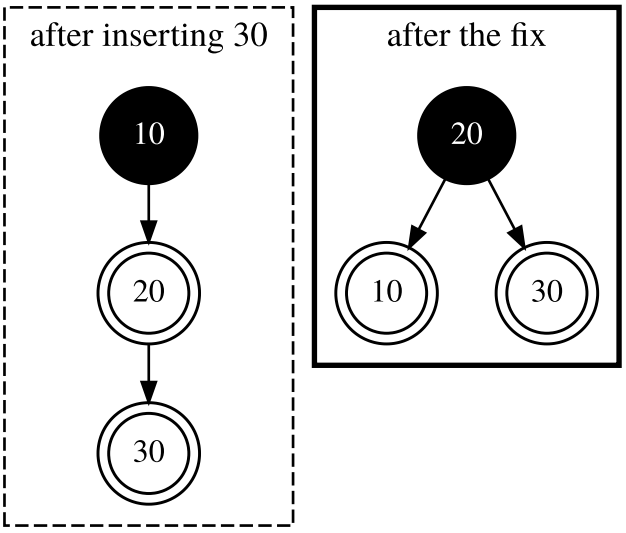
3. Core Explanation / Working

Insertion Fix-up Cases (z = new red node, p = parent, u = uncle, g = grandparent)

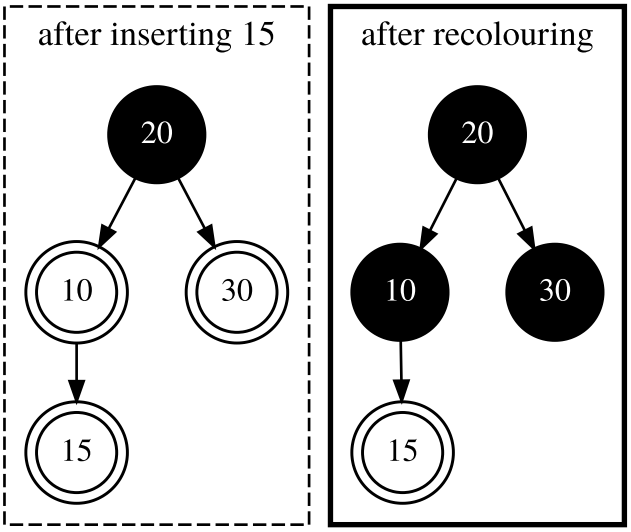
Case	Condition	Action
0	z is the root	Colour z BLACK
-	p is BLACK	No violation, done
1	p is RED and **uncle u is RED**	**Recolour:** p and u BLACK, g RED. Repeat the check at g
2	p RED, u BLACK, z is an **inner** child (triangle: LR or RL)	**Rotate at p** to turn it into Case 3
3	p RED, u BLACK, z is an **outer** child (line: LL or RR)	**Rotate at g**, swap the colours of p and g

Example: insert **10, 20, 30, 15, 12** into an empty Red-Black Tree (R = red, B = black)

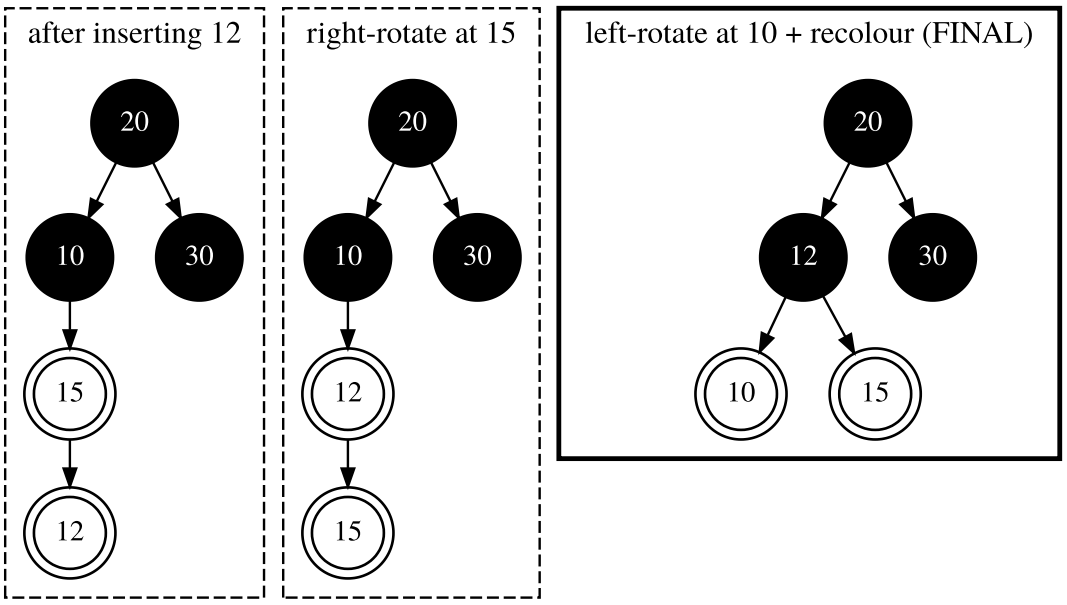
Notation in the figures: filled circle = BLACK node, double circle = RED node.



Insert 30: Case 3 (RR line). Left-rotate at 10 and swap the colours of 10 and 20



Insert 15: Case 1 (uncle 30 is RED). Recolour 10 and 30 BLACK, 20 RED, then the root becomes BLACK again



Insert 12: RL triangle. Case 2 (right-rotate at 15), then Case 3 (left-rotate at 10, recolour 12 BLACK and 10 RED)

Final tree verification:

- The root 20 is BLACK.
- No red node has a red child (10R and 15R have NIL children).
- Black-height from the root is 2 on every path: 20-12-NIL, 20-12-10-NIL, 20-30-NIL.

4. Key Characteristics - Red-Black Tree vs. AVL Tree

Basis	Red-Black Tree	AVL Tree
Balance criterion	Colour rules (black-height)	Height difference ≤ 1
Max height	2 log2(n + 1)	about 1.44 log2 n
Rotations per insert	**At most 2**	Up to 2 (but more rebalancing checks)
Rotations per delete	**At most 3**	O(log n)
Search speed	Slightly slower	**Faster** (more strictly balanced)
Best for	**Insert/delete-heavy** workloads	**Search-heavy** workloads
Used in	Java TreeMap, C++ `std::map`, Linux CFS	Databases needing fast lookups

5. Advantages & Disadvantages

Advantages

- **Guaranteed O(log n)** for search, insert and delete.
- **Few rotations** (at most 2 per insertion), so updates are fast.
- Only **one extra bit** (the colour) is stored per node.

Disadvantages

- **Complex** implementation, with many fix-up cases, especially for deletion.

- Less strictly balanced than an AVL tree, so **lookups are slightly slower**.

6. Conclusion / Application

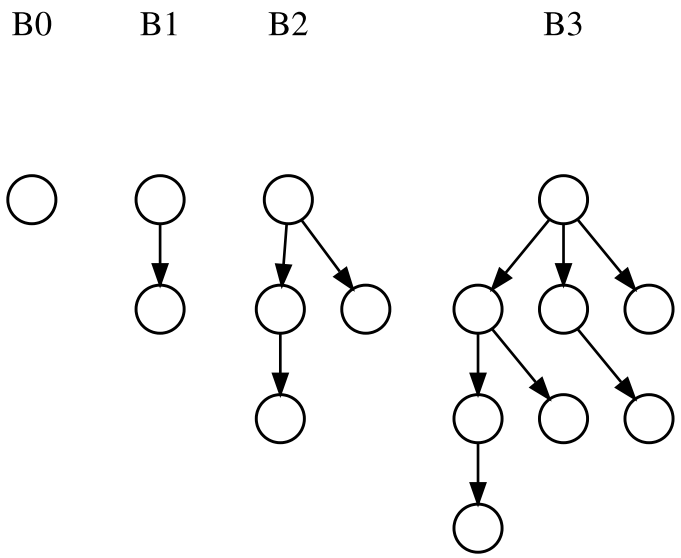
Red-Black Trees are used in **Java's TreeMap and TreeSet, C++ STL `map/set`, and the Linux kernel's Completely Fair Scheduler (CFS)**.

Q4. Explain the structure, properties and merge (union) operation of a Binomial Heap. (7 Marks)

1. Introduction / Definition

A **Binomial Heap** is a **mergeable heap**: a collection of **binomial trees** in which each tree is **min-heap ordered** and there is **at most one binomial tree of each degree**. It supports **Union (merge) in O(log n)** time, compared with O(n) for a binary heap.

2. Diagram - Binomial Trees B0 to B3



Binomial trees B0 to B3: Bk has 2^k nodes and height k; it is formed by linking two B(k−1) trees

3. Core Explanation / Working

(a) Properties of a Binomial Tree Bk

1. It has exactly **2^k nodes**.
2. Its **height is k**.
3. There are exactly **C(k, i)** (the binomial coefficient) nodes at **depth i**. This is where the name comes from.
4. The **root has degree k**, which is the largest degree in the tree. Its children, from left to right, are the roots of **B(k−1), B(k−2), …, B1, B0**.

(b) Properties of a Binomial Heap H

1. Every binomial tree in H is **min-heap ordered** (key of a node ≥ key of its parent).

2. For each degree k, there is **at most one** binomial tree in H.
3. An n-node heap has at most **⌊log2 n⌋ + 1** trees, matching the **1-bits of n in binary**. Example: n = 13 = 1101 (binary), so H has trees **B3, B2 and B0**.
4. The roots are kept in a **root list** (a linked list), sorted by **increasing degree**.

(c) Node Structure

parent	key	degree	child	sibling
pointer to the parent	the key value	number of children	pointer to the leftmost child	pointer to the right sibling

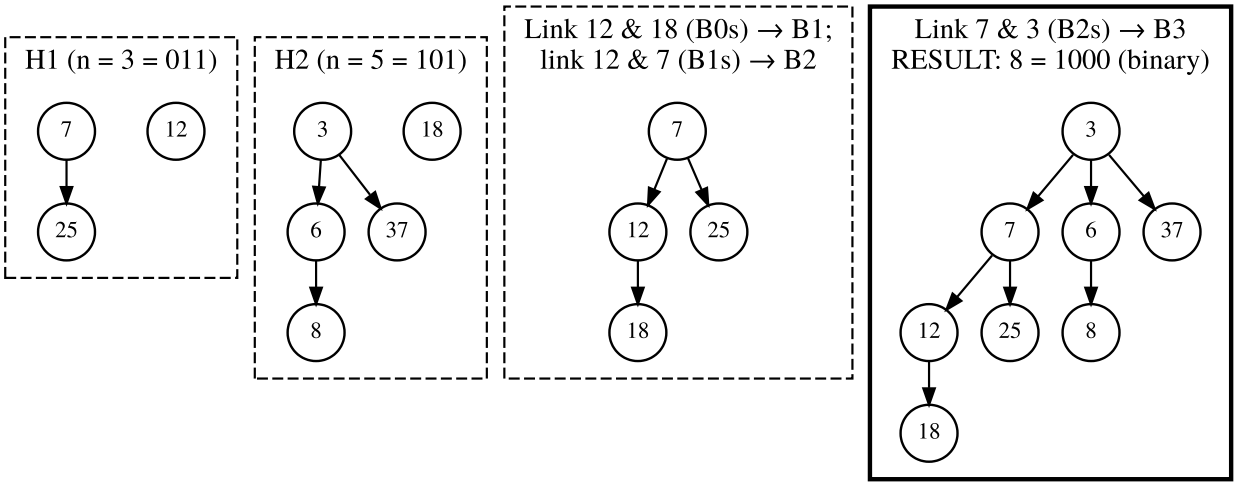
(d) BINOMIAL-LINK(y, z): makes y a child of z, when both are roots of B(k−1) trees and z.key ≤ y.key

```
y.parent  = z
y.sibling = z.child
z.child   = y
z.degree  = z.degree + 1          // O(1)
```

(e) Union (Merge) Operation, BINOMIAL-HEAP-UNION(H1, H2)

1. **Merge** the root lists of H1 and H2 into one list **sorted by degree**, as in merge sort.
2. Scan the list and **link trees of equal degree**. This works like **binary addition with carry**:
   - If the degrees of x and next-x differ, move on.
   - If three trees in a row have the same degree, move on (link the last two).
   - If x.degree = next-x.degree, **link** them, so the root with the **smaller key** becomes the parent.

(f) Worked Example



BINOMIAL-HEAP-UNION of H1 (3 nodes) and H2 (5 nodes)

Step	Root list (by degree)	Action
1. Merge	12(B0) → 18(B0) → 7(B1) → 3(B2)	Merge both root lists, sorted by degree
2	12(B1) → 7(B1) → 3(B2)	Link 12 and 18 (both B0); 12 < 18, so 12 is the root
3	7(B2) → 3(B2)	Link 12 and 7 (both B1); 7 < 12, so 7 is the root
4	3(B3)	Link 7 and 3 (both B2); 3 < 7, so 3 is the root

4. Key Characteristics - Binary Heap vs. Binomial Heap (Time Complexities)

Operation	Binary Heap	Binomial Heap
Make-Heap	O(1)	O(1)
Insert	O(log n)	O(log n) (O(1) amortized)
Find Minimum	**O(1)**	O(log n)
Extract-Min	O(log n)	O(log n)
Union (Merge)	**O(n)**	**O(log n)**
Decrease-Key	O(log n)	O(log n)
Delete	O(log n)	O(log n)

5. Advantages & Disadvantages

Advantages

- **Fast union in O(log n)**, which makes it ideal as a mergeable priority queue.
- Its structure relates directly to **binary numbers**, so operations are easy to analyse.
- Insert takes **O(1) amortized** time.

Disadvantages

- **Find-min takes O(log n)** (the whole root list is scanned) unless a min pointer is kept.
- **Complex pointer structure** (parent, child, sibling) and more memory per node.

6. Conclusion / Application

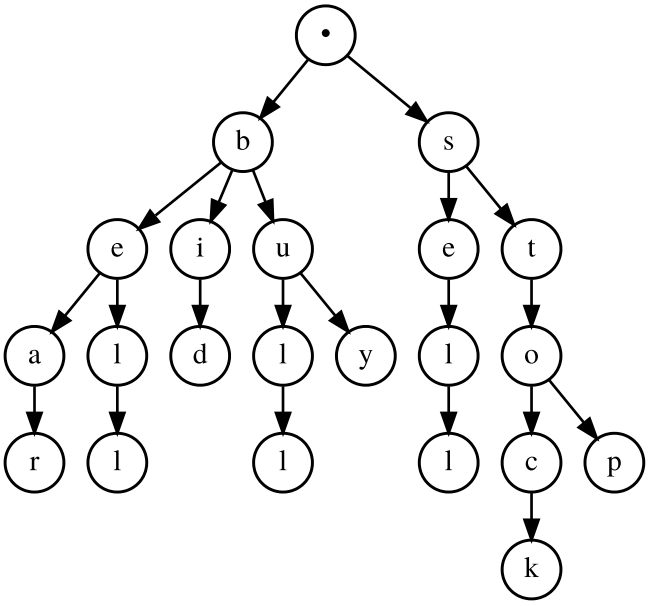
Binomial heaps are used as **mergeable priority queues** in **graph algorithms (Prim's, Dijkstra's), OS process scheduling and discrete-event simulation**, and they are the basis of Fibonacci heaps.

Q5. Construct a compressed Trie for a given set of strings. (7 Marks)

1. Introduction / Definition

A **Trie (prefix tree)** is a tree-based data structure for storing a set of strings, where **each edge is labelled with a character** and each path from the root spells a string. A **Compressed Trie (Radix / Patricia Trie)** is a trie in which **every chain of nodes with a single child is merged into one edge** labelled with a **substring**. As a result, every internal node has **at least 2 children**.

2. Diagram - Standard Trie for S = {bear, bell, bid, bull, buy, sell, stock, stop}



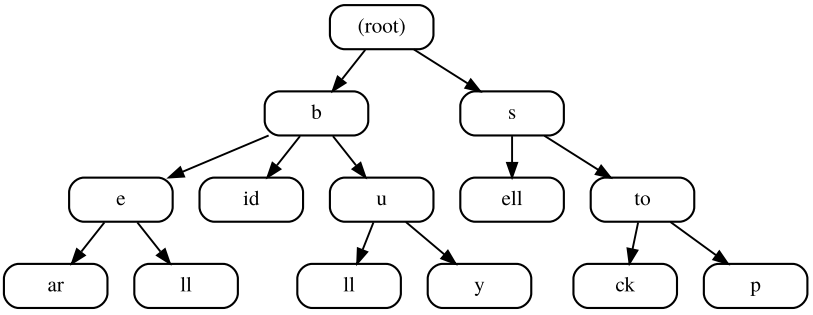
Standard trie for {bear, bell, bid, bull, buy, sell, stock, stop} (22 nodes)

3. Core Explanation / Working

Steps to build the compressed trie

1. **Build the standard trie:** insert every string character by character, sharing common prefixes.
2. **Find redundant chains:** look for internal nodes that have **exactly one child** and are not the end of a word.
3. **Compress:** merge each such chain into **one edge**, labelled with the **concatenated substring**.
4. **Check:** every internal node now has **at least 2 children**, and the number of nodes is **O(s)**, where s is the number of strings.

Compressed Trie for S = {bear, bell, bid, bull, buy, sell, stock, stop}



Compressed trie: single-child chains merged into one edge (14 nodes)

Reading root to leaf: b+e+ar = bear, b+e+ll = bell, b+id = bid, b+u+ll = bull, b+u+y = buy, s+ell = sell, s+to+ck = stock, s+to+p = stop.

Compression actions taken

- `a → r` became **“ar”**, and `l → l` (under e) became **“ll”**.
- `i → d` became **“id”**. `l → l` (under u) became **“ll”**.

- `e → l → l` (under s) became **“ell”**.
- `t → o` became **“to”**, and `c → k` became **“ck”**.

Node count: standard trie = **22 nodes** (including the root), compressed trie = **14 nodes**.

Compact representation (to save space): each edge label is stored as a triple **(i, j, k)**, meaning the substring `S[i][j..k]`, instead of storing the characters.

4. Key Characteristics - Standard Trie vs. Compressed Trie

Basis	Standard Trie	Compressed Trie
Edge label	Single character	**Substring**
Internal node degree	Can be 1	**At least 2**
Number of nodes	O(total length of all strings)	**O(s)** (s = number of strings)
Space	High	**Low**
Search time	O(m) (m = pattern length)	O(m)
Insertion	Simple	More complex (edge splitting)

5. Advantages & Disadvantages

Advantages

- **Saves memory** by removing single-child chains.
- **Fast prefix search** in O(m), independent of the number of stored strings.
- It is the basis of **suffix trees** for advanced pattern matching.

Disadvantages

- **Insertion and deletion are more complex**, because edges must be split or merged.
- Edge labels store substrings, which adds **implementation complexity**.

6. Conclusion / Application

Compressed tries are used in **autocomplete and spell checkers, IP routing tables (longest prefix match), and suffix trees** for DNA sequence analysis.

Unit 3: Greedy Algorithms & Graph Theory

Q1. Short Definitions (2 Marks Each)

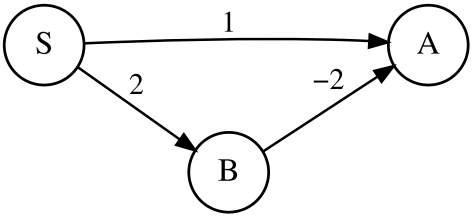
(a) What is the greedy choice property?

- The **greedy choice property** means that a **globally optimal solution can be reached by making a locally optimal (greedy) choice** at each step, without reconsidering earlier choices.
- Together with **optimal substructure**, it guarantees that a greedy algorithm gives the correct answer.

- *Example:* in the **Activity Selection** problem, always choosing the activity that **finishes earliest** leaves the most time for the rest, and this leads to the optimal solution. (Counter-example: greedy coin change with coins {1, 3, 4} for amount 6 gives 4+1+1 = 3 coins, but the optimum is 3+3 = 2 coins. The property does not hold there.)

(b) When does Dijkstra's algorithm fail to find the shortest path?

- Dijkstra's algorithm **fails when the graph has negative-weight edges**. It **finalizes** a vertex as soon as it is extracted from the priority queue, assuming its distance can never decrease later. A negative edge breaks this assumption.



Dijkstra fails with a negative edge: it finalizes d(A) = 1, but the true shortest distance is $S \to B \to A = 2 + (-2) = 0$

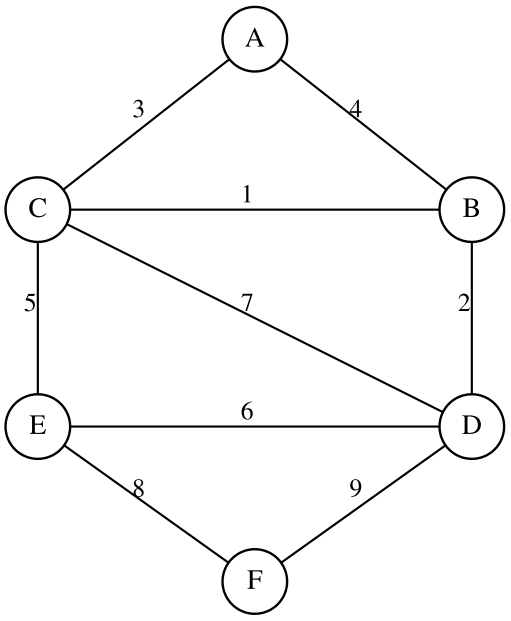
- **Solution:** use the **Bellman-Ford algorithm**, which handles negative edges and also detects negative cycles.

Q2. Trace Kruskal's and Prim's algorithms to generate a Minimum Spanning Tree (MST) for a given weighted graph. (7 Marks)

1. Introduction / Definition

A **Minimum Spanning Tree (MST)** of a connected, weighted, undirected graph G(V, E) is a **spanning tree** (it connects all V vertices with exactly V − 1 edges and has no cycles) whose **total edge weight is minimum**. **Kruskal's** and **Prim's** are **greedy algorithms** for finding the MST.

2. Block Diagram - Given Graph and Algorithm Flow



Given weighted graph (6 vertices, 9 edges)

3. Core Explanation / Working

(A) Kruskal's Algorithm

Algorithm

1. Sort all edges in **non-decreasing order** of weight.
2. Make each vertex a **separate set** (Union-Find).
3. For each edge (u, v) in sorted order: if **FIND(u) ≠ FIND(v)**, add the edge to the MST and **UNION(u, v)**. Otherwise **reject** it, because it would form a cycle.
4. Stop when the MST has **V – 1 edges**.

Trace

Step	Edge	Weight	Cycle?	Action	Components after the step
1	B-C	1	No	**Accept**	{B,C} {A} {D} {E} {F}
2	B-D	2	No	**Accept**	{B,C,D} {A} {E} {F}
3	A-C	3	No	**Accept**	{A,B,C,D} {E} {F}
4	A-B	4	Yes (A, B same set)	Reject	-
5	C-E	5	No	**Accept**	{A,B,C,D,E} {F}
6	D-E	6	Yes	Reject	-
7	C-D	7	Yes	Reject	-
8	E-F	8	No	**Accept**	{A,B,C,D,E,F} (5 edges, stop)

(B) Prim's Algorithm (starting from vertex A)

Algorithm

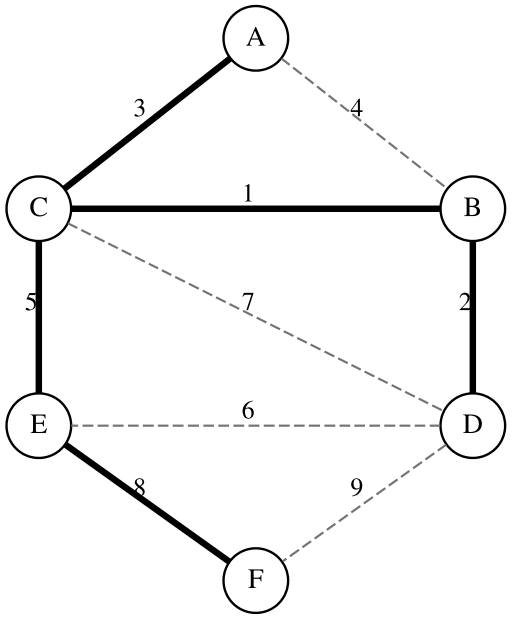
1. Start with any vertex, and set key[start] = 0 and every other key[v] = ∞.

2. Repeatedly pick the **non-tree vertex with the minimum key** (the cheapest edge connecting it to the tree).
3. Add it to the tree and **update the keys** of its neighbours.
4. Stop when all vertices are in the tree.

Trace

Step	Tree vertices	Candidate edges (crossing the cut)	Selected edge
1	{A}	A-B 4, **A-C 3**	**A-C (3)**
2	{A, C}	A-B 4, **C-B 1**, C-D 7, C-E 5	**C-B (1)**
3	{A, C, B}	**B-D 2**, C-D 7, C-E 5	**B-D (2)**
4	{A, C, B, D}	**C-E 5**, D-E 6, D-F 9	**C-E (5)**
5	{A, C, B, D, E}	**E-F 8**, D-F 9	**E-F (8)**
6	{A, B, C, D, E, F}	-	Done

Resulting MST (same from both algorithms)



Minimum spanning tree (bold edges), total weight = 1 + 2 + 3 + 5 + 8 = 19

4. Key Characteristics - Kruskal vs. Prim

Basis	Kruskal's Algorithm	Prim's Algorithm
Approach	**Edge-based**: picks the globally smallest safe edge	**Vertex-based**: grows one tree from a start vertex
Intermediate result	A **forest** (several trees)	Always **one connected tree**
Data structure	Sorting + **Union-Find (disjoint set)**	**Min-priority queue** (binary/Fibonacci heap)
Time complexity	**O(E log E)** = O(E log V)	O(E log V) with binary heap; O(E + V log V) with Fibonacci heap
Best for	**Sparse** graphs	**Dense** graphs
Start vertex	Not needed	Needed

5. Advantages & Disadvantages

Advantages

- Both **always give an optimal MST** (proved by the cut property).
- Kruskal is **simple to implement** with Union-Find, and Prim is **efficient on dense graphs**.
- Both run in **polynomial time**.

Disadvantages

- Kruskal needs **all edges sorted first**, which is slow for very dense graphs.
- Prim needs a **priority queue** with decrease-key, which is more complex to code.
- Both work only on **undirected, connected** graphs. For directed graphs, Edmonds' algorithm is needed.

6. Conclusion / Application

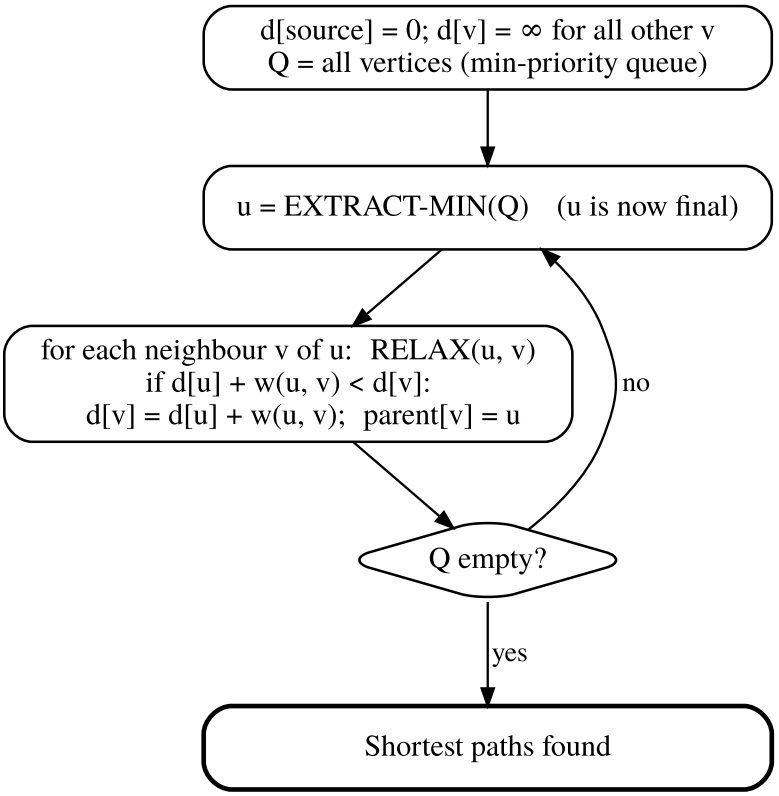
MST algorithms are used to design **minimum-cost networks: laying telephone or fibre cables, electrical grids and road networks**, and in **cluster analysis**.

Q3. Find the shortest path using Dijkstra's algorithm on a given graph. (7 Marks)

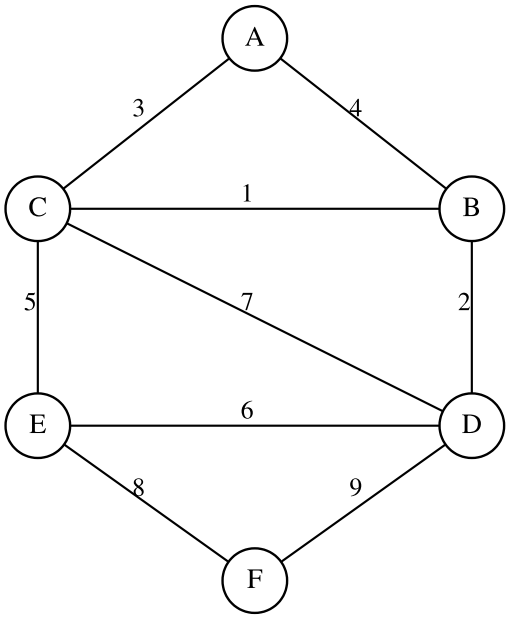
1. Introduction / Definition

Dijkstra's algorithm is a **greedy single-source shortest path** algorithm. It finds the shortest distance from a source vertex to **all other vertices** in a weighted graph with **non-negative edge weights**. It repeatedly selects the unvisited vertex with the **smallest tentative distance** and **relaxes** its outgoing edges.

2. Flowchart and Graph



Dijkstra's algorithm



Graph used (source = A)

3. Core Explanation / Working

Algorithm

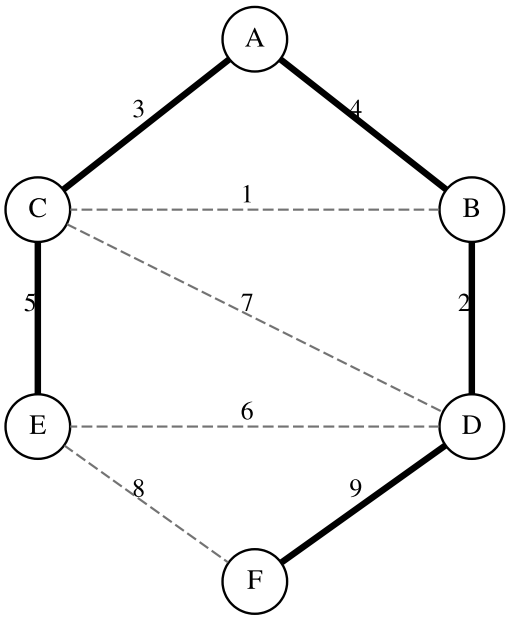
```
DIJKSTRA(G, w, s)
1. for each vertex v:  d[v] = INF,  parent[v] = NIL
2. d[s] = 0
3. S = EMPTY ;  Q = V            // min-priority queue on d
4. while Q is not empty
5.     u = EXTRACT-MIN(Q)
6.     S = S U {u}
7.     for each vertex v adjacent to u
8.         if d[v] > d[u] + w(u, v)          // RELAX
9.             d[v] = d[u] + w(u, v)
10.            parent[v] = u
```

Trace (source = A). Bold = vertex selected (finalized) in that iteration.

Iteration	Selected u	d[A]	d[B]	d[C]	d[D]	d[E]	d[F]	Relaxations done
Init	-	0	∞	∞	∞	∞	∞	-
1	**A** (0)	**0**	4	3	∞	∞	∞	B = 0+4, C = 0+3
2	**C** (3)	0	4	**3**	10	8	∞	B: 3+1 = 4 (no change), D = 3+7, E = 3+5
3	**B** (4)	0	**4**	3	6	8	∞	D: 4+2 = 6 < 10, update
4	**D** (6)	0	4	3	**6**	8	15	E: 6+6 = 12 > 8 (no change), F = 6+9
5	**E** (8)	0	4	3	6	**8**	15	F: 8+8 = 16 > 15 (no change)
6	**F** (15)	0	4	3	6	8	**15**	-

Final shortest paths from A

Vertex	Distance	Path (from the parent pointers)
A	0	A
B	4	A → B (A → C → B is also 4)
C	3	A → C
D	6	A → B → D
E	8	A → C → E
F	15	A → B → D → F



Shortest-path tree from A (bold edges); distances A0, B4, C3, D6, E8, F15

Complexity

- With an array: **O(V²)**.
- With a binary heap: **O((V + E) log V)**.
- With a Fibonacci heap: **O(E + V log V)**.

4. Key Characteristics - Dijkstra vs. Bellman-Ford

Basis	Dijkstra	Bellman-Ford
Strategy	**Greedy**	**Dynamic programming** (relax all edges V − 1 times)
Negative edge weights	**Not allowed**	Allowed
Negative cycle detection	No	**Yes**
Time complexity	O((V + E) log V)	O(V · E)
Speed	**Faster**	Slower

5. Advantages & Disadvantages

Advantages

- **Efficient**, especially with a heap, and finds paths to **all vertices** in one run.
- Simple greedy logic that is **guaranteed optimal** for non-negative weights.
- Works on both directed and undirected graphs.

Disadvantages

- **Fails with negative edge weights.**
- Explores in all directions (it is “blind”), so A* search is faster for a single target.

6. Conclusion / Application

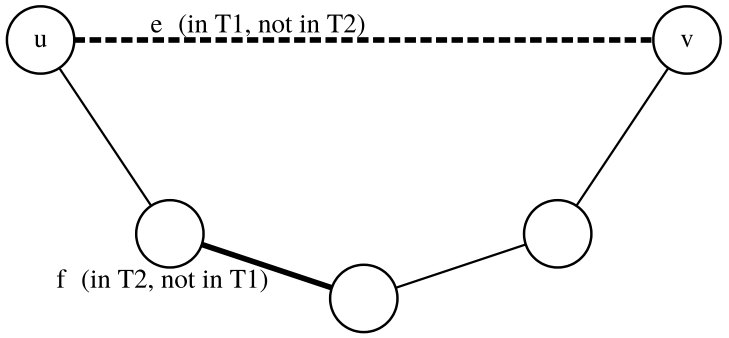
Dijkstra's algorithm is used in **Google Maps and GPS navigation, network routing protocols like OSPF, and flight-route planning**.

Q4. Prove that if the edge weights of a connected undirected graph are distinct, it has a unique MST. (7 Marks)

1. Introduction / Definition

Theorem: if G = (V, E) is a **connected, undirected, weighted graph** in which **all edge weights are distinct**, then G has **exactly one Minimum Spanning Tree**. The proof is **by contradiction**, using the **cycle property** and an **exchange argument**.

2. Diagram - Exchange Argument



path in T2 from u to v

Exchange argument: adding e to T2 creates a cycle; removing the heavier edge f gives a lighter spanning tree

$$w(e) < w(f) \quad \Rightarrow \quad w(T_2') = w(T_2) + w(e) - w(f) < w(T_2) \quad \text{(contradiction)}$$

3. Core Explanation / Working - The Proof

Step 1 - Assumption

- Suppose, for contradiction, that G has **two different MSTs, T1 and T2**, with **w(T1) = w(T2)** (both minimum).

Step 2 - Choose the critical edge

- Since T1 ≠ T2 and both have V − 1 edges, the **symmetric difference T1 ⊕ T2** (edges in exactly one of the two trees) is **non-empty**.
- Let **e = (u, v)** be the edge with the **minimum weight** in T1 ⊕ T2. This edge is **unique** because the weights are distinct.
- Without loss of generality, assume **e ∈ T1 and e ∉ T2**.

Step 3 - Add e to T2

- T2 is a spanning tree, so it already contains a **unique path from u to v**.
- Adding e to T2 creates **exactly one cycle C**.

Step 4 - Find an edge f on the cycle that is not in T1

- The cycle C cannot be **entirely inside T1**, because T1 is a tree and has no cycles.
- So C contains at least one edge **f ≠ e** with **f ∈ T2 and f ∉ T1**.
- Therefore f ∈ T1 ⊕ T2.

Step 5 - Compare weights

- e is the **minimum** edge in T1 ⨁ T2, and f is also in T1 ⨁ T2.
- The weights are **distinct**, so **w(e) < w(f)** (strictly).

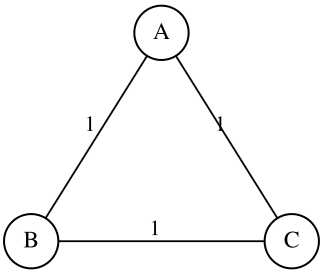
Step 6 - Build a lighter tree

- Let **T2' = T2 ∪ {e} − {f}**.
- Removing f breaks the only cycle C, so T2' is **connected and acyclic with V − 1 edges**. It is a **spanning tree**.
- **w(T2') = w(T2) + w(e) − w(f) < w(T2)**

Step 7 - Contradiction

- T2' has **smaller weight than T2**, which contradicts the assumption that T2 is a **minimum** spanning tree.
- So the assumption was false, and **T1 = T2**.
- **Hence, the MST is unique when all edge weights are distinct. (Proved)**

Note (the converse is false): a graph with repeated weights **may still have a unique MST**, but uniqueness is then not guaranteed. For example, a triangle with all edges of weight 1 has **3 different MSTs**.



Equal weights: this triangle has 3 different MSTs (A-B + A-C, A-B + B-C, A-C + B-C), each of weight 2

4. Key Characteristics - Properties Used in the Proof

Property	Statement	Role in the Proof
Tree property	A spanning tree has V − 1 edges and a unique path between any two vertices	Adding e creates exactly one cycle
Cycle property	The heaviest edge on a cycle is not in any MST (with distinct weights)	f is heavier than e, so it can be swapped out
Cut property	The lightest edge crossing any cut is in every MST	Alternative proof method
Exchange argument	Replace one edge with a lighter one, keeping the tree structure	Gives the lighter tree T2'

5. Advantages & Disadvantages (of the result)

Advantages

- **Guarantees** that Kruskal's and Prim's algorithms give the **same MST** whatever the tie-breaking or start vertex.
- Gives a **simple test**: if the weights are distinct, there is no need to search for other MSTs.
- The exchange argument is a **general proof technique** for greedy algorithms.

Disadvantages / Limitations

- The condition is **sufficient but not necessary**: graphs with repeated weights may also have unique MSTs.
- It applies only to **connected, undirected** graphs.

6. Conclusion / Application

This theorem ensures **deterministic network design**: in cable or road planning with distinct costs, every correct MST algorithm gives the **same optimal layout**.

Unit 4: Dynamic Programming

Q1. Short Definitions (2 Marks Each)

(a) State the Principle of Optimality.

- **Bellman's Principle of Optimality:** *"An optimal sequence of decisions has the property that, whatever the initial state and first decision are, the remaining decisions must form an optimal sequence with respect to the state that results from the first decision."*
- In simple words: **an optimal solution to a problem contains optimal solutions to its sub-problems** (optimal substructure).
- *Example:* if the shortest path from A to C passes through B, then the A-to-B part and the B-to-C part must **also be shortest paths**.

(b) Contrast Divide and Conquer with Dynamic Programming.

Basis	Divide and Conquer	Dynamic Programming
Sub-problems	**Independent** (do not overlap)	**Overlapping** (the same sub-problem repeats)
Re-computation	The same sub-problem may be solved again and again	Each sub-problem is solved **once** and stored (table / memoization)
Approach	Top-down (recursive)	Bottom-up (tabulation) or top-down with memoization
Memory	Less (no table)	More (DP table)
Efficiency on overlapping problems	Exponential (for example naive Fibonacci O(2^n))	Polynomial (Fibonacci O(n))
Examples	Merge Sort, Quick Sort, Binary Search	0/1 Knapsack, LCS, Matrix Chain, Floyd-Warshall

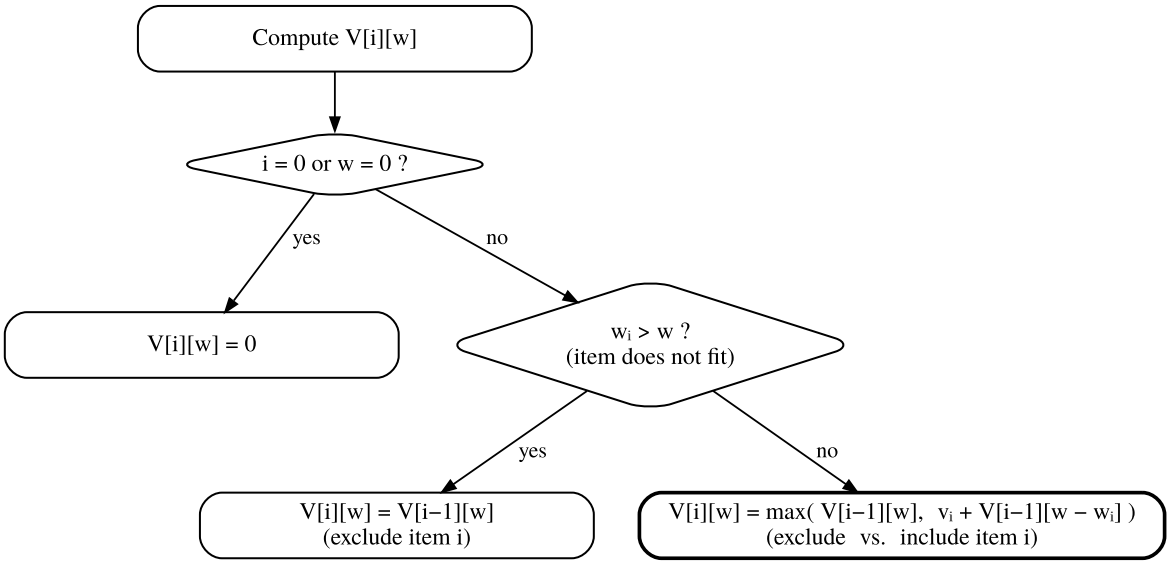
---

Q2. Solve the 0/1 Knapsack problem using the DP table method. (7 Marks)

1. Introduction / Definition

In the **0/1 Knapsack problem**, we have n items, each with a **weight wi** and a **value vi**, and a knapsack of **capacity W**. We must choose items to **maximise the total value** while the total weight stays **≤ W**. Each item is either **taken completely (1) or left (0)**. Fractions are not allowed. Dynamic programming solves it in **O(n·W)** time.

2. Flowchart - DP Recurrence



Filling one cell of the 0/1 Knapsack table

3. Core Explanation / Working

Recurrence

$$V[i][w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ V[i-1][w] & \text{if } w_i > w \\ \max\left(V[i-1][w],\ v_i + V[i-1][w-w_i]\right) & \text{otherwise} \end{cases}$$

Problem: capacity **W = 5**, n = 4 items.

Item	1	2	3	4
Weight (wi)	2	3	4	5
Value (vi)	3	4	5	6

DP Table V[i][w]

i w	0	1	2	3	4	5
0 (no item)	0	0	0	0	0	0
1 (w=2, v=3)	0	0	3	3	3	3
2 (w=3, v=4)	0	0	3	4	4	**7**
3 (w=4, v=5)	0	0	3	4	5	7
4 (w=5, v=6)	0	0	3	4	5	**7**

Sample calculations

- V[1][2] = max(V[0][2], 3 + V[0][0]) = max(0, 3) = **3**
- V[2][3] = max(V[1][3], 4 + V[1][0]) = max(3, 4) = **4**
- V[2][5] = max(V[1][5], 4 + V[1][2]) = max(3, 4 + 3) = **7**
- V[3][4] = max(V[2][4], 5 + V[2][0]) = max(4, 5) = **5**
- V[3][5] = max(V[2][5], 5 + V[2][1]) = max(7, 5) = **7**
- V[4][5] = max(V[3][5], 6 + V[3][0]) = max(7, 6) = **7**

Traceback (which items were chosen?) Start at V[4][5] = 7 and move up.

Check	Comparison	Decision	Remaining capacity
i = 4	V[4][5] = 7 = V[3][5] = 7	Item 4 **not taken**	5
i = 3	V[3][5] = 7 = V[2][5] = 7	Item 3 **not taken**	5
i = 2	V[2][5] = 7 ≠ V[1][5] = 3	Item 2 **TAKEN**	5 − 3 = 2
i = 1	V[1][2] = 3 ≠ V[0][2] = 0	Item 1 **TAKEN**	2 − 2 = 0

Answer: take **items 1 and 2**. Total weight = 2 + 3 = 5, **maximum value = 3 + 4 = 7**. Solution vector **x = (1, 1, 0, 0)**.

Complexity: time **O(n·W)**, space **O(n·W)** (this can be reduced to O(W) with a 1-D array). This is **pseudo-polynomial**, because it depends on the numeric value of W.

4. Key Characteristics - 0/1 Knapsack vs. Fractional Knapsack

Basis	0/1 Knapsack	Fractional Knapsack
Item division	Not allowed (take all or none)	Allowed (take fractions)
Technique	**Dynamic Programming**	**Greedy** (by value/weight ratio)
Greedy works?	No	Yes
Time complexity	O(n·W)	O(n log n)
Problem class	NP-complete (decision version)	Polynomial (P)

5. Advantages & Disadvantages

Advantages

- **Guarantees the optimal solution**, unlike greedy.
- **Avoids recomputation** of overlapping sub-problems.
- The table also lets us **trace back the chosen items**.

Disadvantages

- **Pseudo-polynomial**: it is impractical when W is very large (for example 10^9).
- Needs **O(n·W) memory** for the full table.
- Works only with **integer weights**.

6. Conclusion / Application

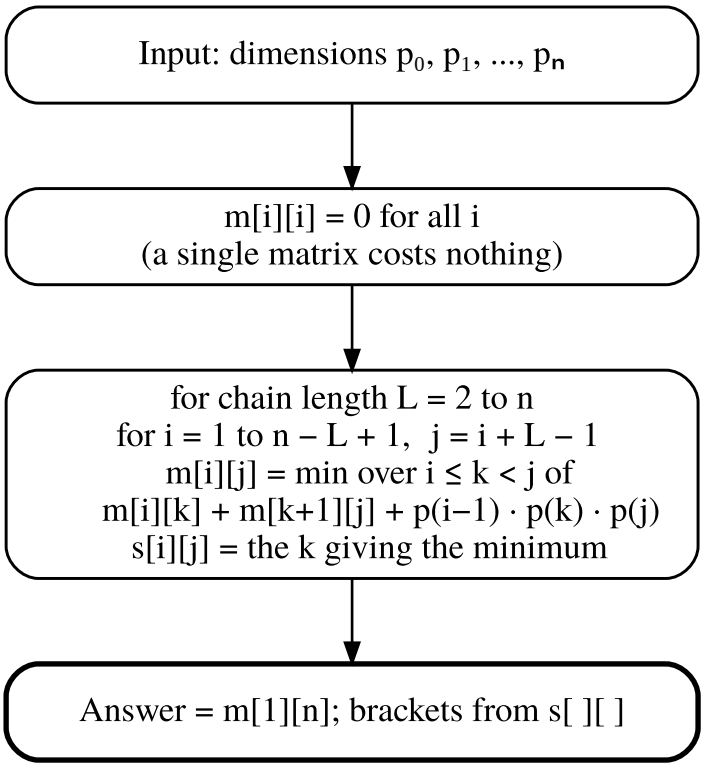
0/1 Knapsack is used in **resource allocation, cargo loading, investment portfolio selection and budget-constrained project selection**.

Q3. Trace the Matrix Chain Multiplication algorithm to find the optimal parenthesization. (7 Marks)

1. Introduction / Definition

The **Matrix Chain Multiplication (MCM)** problem: given a chain of matrices A1, A2, …, An, where Ai has dimension **p(i−1) × p(i)**, find the **parenthesization** that **minimises the number of scalar multiplications**. Matrix multiplication is **associative**, so every order gives the same product but a different cost. DP solves this in **O(n^3)** time.

2. Flowchart - Algorithm



Matrix Chain Multiplication (bottom-up DP)

3. Core Explanation / Working

Recurrence

$$m[i][j] = \begin{cases} 0 & \text{if } i = j \\ \min\limits_{i \le k < j} \{m[i][k] + m[k+1][j] + p_{i-1} p_k p_j\} & \text{if } i < j \end{cases}$$

Problem: 4 matrices

Matrix	A1	A2	A3	A4
Dimension	5 × 4	4 × 6	6 × 2	2 × 7

The dimension sequence is **p = <p0, p1, p2, p3, p4> = <5, 4, 6, 2, 7>**.

Step 1 - Chain length L = 1: m[1][1] = m[2][2] = m[3][3] = m[4][4] = 0

Step 2 - Chain length L = 2

- m[1][2] = p0·p1·p2 = 5·4·6 = **120** (s = 1)
- m[2][3] = p1·p2·p3 = 4·6·2 = **48** (s = 2)
- m[3][4] = p2·p3·p4 = 6·2·7 = **84** (s = 3)

Step 3 - Chain length L = 3

- **m[1][3]:**
  - k = 1: m[1][1] + m[2][3] + p0·p1·p3 = 0 + 48 + 5·4·2 = **88**
  - k = 2: m[1][2] + m[3][3] + p0·p2·p3 = 120 + 0 + 5·6·2 = 180
  - m[1][3] = **88**, s[1][3] = **1**
- **m[2][4]:**
  - k = 2: m[2][2] + m[3][4] + p1·p2·p4 = 0 + 84 + 4·6·7 = 252
  - k = 3: m[2][3] + m[4][4] + p1·p3·p4 = 48 + 0 + 4·2·7 = **104**
  - m[2][4] = **104**, s[2][4] = **3**

Step 4 - Chain length L = 4

- **m[1][4]:**
  - k = 1: m[1][1] + m[2][4] + p0·p1·p4 = 0 + 104 + 5·4·7 = 244
  - k = 2: m[1][2] + m[3][4] + p0·p2·p4 = 120 + 84 + 5·6·7 = 414
  - k = 3: m[1][3] + m[4][4] + p0·p3·p4 = 88 + 0 + 5·2·7 = **158**
  - m[1][4] = **158**, s[1][4] = **3**

Final Tables

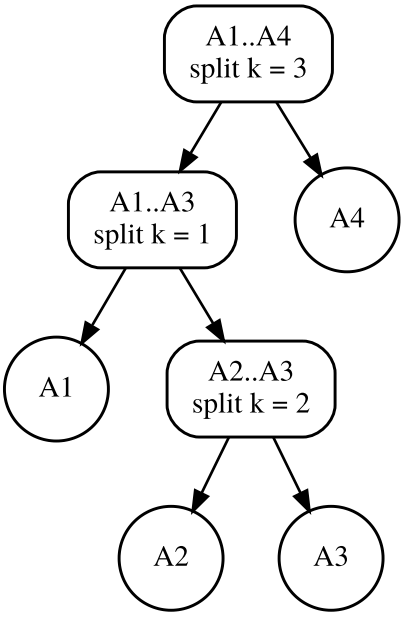
m[i][j] (cost table)

i j	1	2	3	4
1	0	120	88	**158**
2		0	48	104
3			0	84
4				0

s[i][j] (split table)

i j	2	3	4
1	1	1	**3**
2		2	3
3			3

Step 5 - Building the optimal parenthesization from s



Parenthesization tree from the s table: ((A1 (A2 A3)) A4)

Optimal parenthesization: ((A1 (A2 A3)) A4) with **minimum cost = 158** scalar multiplications. (For comparison, the left-to-right order ((A1 A2) A3) A4 costs 120 + 60 + 70 = 250.)

Complexity: time **O(n³)**, space **O(n²)**.

4. Key Characteristics - Brute Force vs. DP for MCM

Basis	Brute Force (try all parenthesizations)	Dynamic Programming
Number of orders checked	Catalan number C(n−1) = Ω(4^n / n^1.5)	Each sub-chain computed once
Time complexity	**Exponential**	**O(n³)**
Space	O(n) recursion	O(n²) tables
Overlapping sub-problems	Recomputed	Stored in m[ ][ ]

5. Advantages & Disadvantages

Advantages

- Reduces exponential work to **polynomial O(n³)**.
- Gives both the **minimum cost** and the **actual bracket order** (through the s table).
- The same DP pattern (“interval DP”) solves **optimal BST and polygon triangulation**.

Disadvantages

- **O(n^2) memory** for the tables.
- **O(n^3)** is slow for very long chains (although faster O(n log n) algorithms exist, they are complex).

6. Conclusion / Application

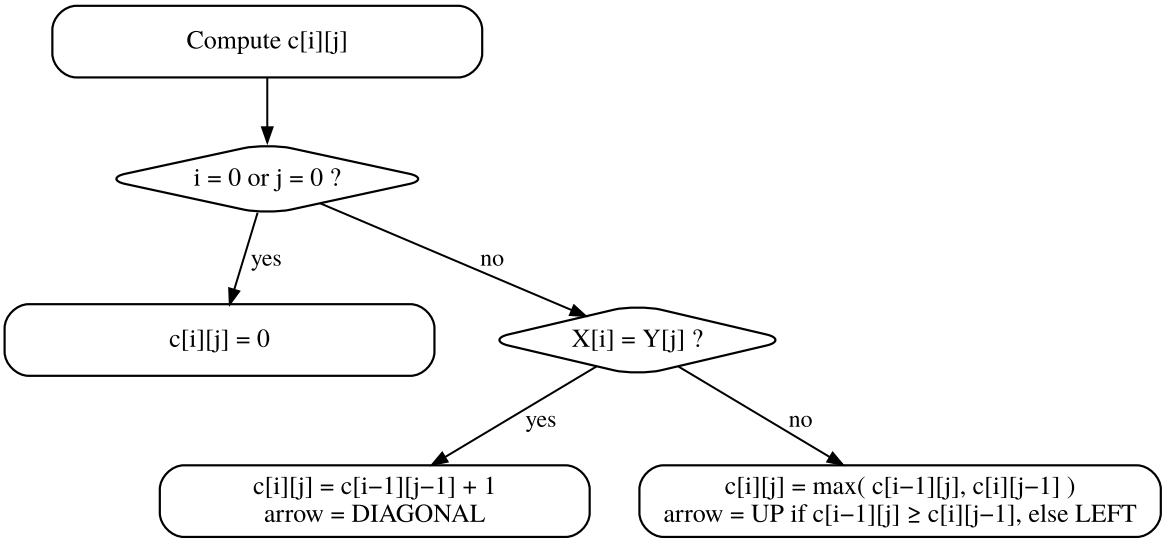
MCM optimization is used in **compilers and query optimizers (join ordering), computer graphics transformation pipelines and scientific computing libraries**.

Q4. Solve the Longest Common Subsequence (LCS) problem for two given strings. (7 Marks)

1. Introduction / Definition

A **subsequence** is a sequence obtained by deleting zero or more characters **without changing the order** of the remaining ones. The **Longest Common Subsequence (LCS)** of two strings X and Y is the **longest sequence that is a subsequence of both**. DP solves it in **O(m·n)** time, where m and n are the string lengths.

2. Flowchart - LCS Recurrence



Filling one cell of the LCS table

3. Core Explanation / Working

Recurrence

$$c[i][j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i-1][j-1] + 1 & \text{if } X_i = Y_j \\ \max\left(c[i-1][j],\ c[i][j-1]\right) & \text{if } X_i \neq Y_j \end{cases}$$

Problem: **X = A B C B D A B** (m = 7), **Y = B D C A B A** (n = 6)

DP Table c[i][j] (D = diagonal, U = up, L = left)

i j	0	1: B	2: D	3: C	4: A	5: B	6: A
0	0	0	0	0	0	0	0
A (1)	0	0 U	0 U	0 U	**1 D**	1 L	1 D
B (2)	0	**1 D**	1 L	1 L	1 U	2 D	2 L
C (3)	0	1 U	1 U	**2 D**	2 L	2 U	2 U
B (4)	0	1 D	1 U	2 U	2 U	**3 D**	3 L
D (5)	0	1 U	2 D	2 U	2 U	3 U	3 U
A (6)	0	1 U	2 U	2 U	3 D	3 U	**4 D**
B (7)	0	1 D	2 U	2 U	3 U	4 D	**4 U**

Sample calculations

- c[1][4]: X1 = A = Y4 = A, so c[0][3] + 1 = **1**
- c[2][5]: X2 = B = Y5 = B, so c[1][4] + 1 = **2**
- c[3][3]: X3 = C = Y3 = C, so c[2][2] + 1 = **2**
- c[4][5]: X4 = B = Y5 = B, so c[3][4] + 1 = **3**
- c[6][6]: X6 = A = Y6 = A, so c[5][5] + 1 = **4**
- c[7][6]: B ≠ A, so max(c[6][6], c[7][5]) = max(4, 4) = **4** (UP)

Traceback from c[7][6] (follow the arrows; every D arrow adds a character)

Cell	X[i], Y[j]	Arrow	Action
(7, 6)	B, A	U	Move up
(6, 6)	A, A	**D**	Output **A**
(5, 5)	D, B	U	Move up
(4, 5)	B, B	**D**	Output **B**
(3, 4)	C, A	L	Move left
(3, 3)	C, C	**D**	Output **C**
(2, 2)	B, D	L	Move left
(2, 1)	B, B	**D**	Output **B**
(1, 0)	-	-	Stop

- Reading the output characters in reverse order gives the LCS **“B C B A”**.

Answer: LCS = “BCBA”, length = 4. (Other LCSs of length 4, such as “BDAB” and “BCAB”, also exist. Different tie-breaking gives a different one.)

Complexity: time **O(m·n)**, space **O(m·n)**. The traceback takes O(m + n).

4. Key Characteristics - LCS vs. Longest Common Substring

Basis	Longest Common Subsequence	Longest Common Substring
Contiguous characters?	**Not required**	**Required**
On mismatch	c[i][j] = max(up, left)	c[i][j] = 0 (reset)
Answer location	c[m][n]	Maximum value anywhere in the table
Example (ABCBDAB, BDCABA)	“BCBA” (length 4)	“AB” / “BD” / “CA” (length 2)

5. Advantages & Disadvantages

Advantages

- Reduces the brute-force **O(2^m · n)** to **O(m·n)**.
- The table gives the **length and the actual subsequence**.
- The **same technique** solves edit distance and sequence alignment.

Disadvantages

- **O(m·n) memory** is large for long strings (Hirschberg’s algorithm reduces it to O(min(m, n))).
- It finds **one** LCS. Listing all of them needs extra work.

6. Conclusion / Application

LCS is used in the **`diff` utility and Git (file comparison), DNA and protein sequence alignment in bioinformatics, and plagiarism detection**.

Unit 5: Backtracking & Complexity Classes

Q1. Short Definitions (2 Marks Each)

(a) Briefly explain the backtracking approach.

- **Backtracking** is a systematic, **depth-first search** of the **state-space tree** of a problem. A solution is built **one component at a time**, and a partial solution is **abandoned (backtracked)** as soon as it is found that it **cannot lead to a valid solution** (it fails a **bounding function**, or constraint).
- Steps: **choose** an option, **check** the constraints, **recurse** if valid, otherwise **undo the choice and try the next option**.
- *Examples:* N-Queens, Sum of Subsets, Graph Colouring, Hamiltonian Cycle, Sudoku.

(b) Define the N-Queens problem.

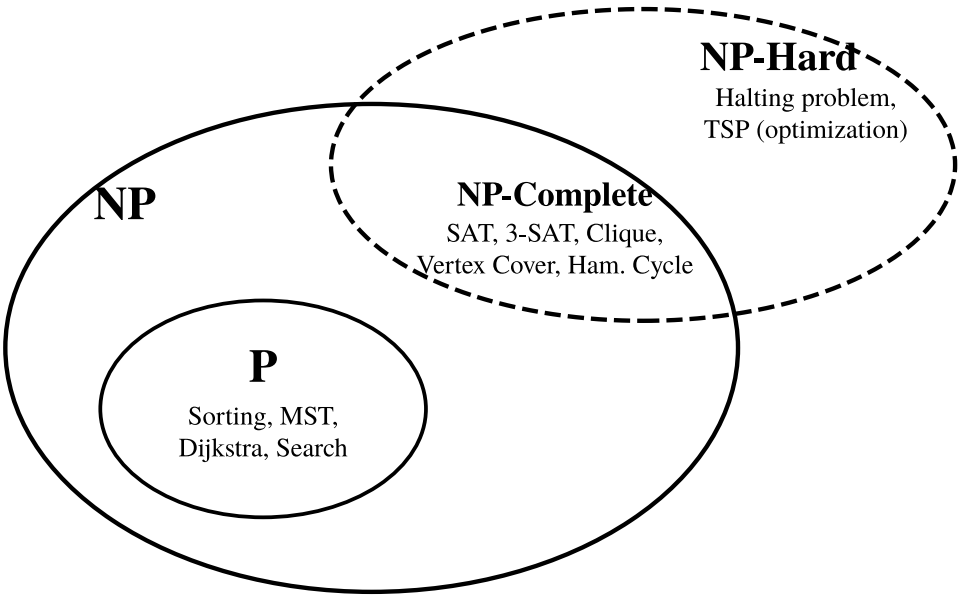
- The **N-Queens problem** is to place **N queens on an N × N chessboard** so that **no two queens attack each other**. That means no two queens share the **same row, the same column or the same diagonal**.
- Queens Qi and Qj (in rows i and j, columns xi and xj) attack each other if **xi = xj** (same column) or **|xi − xj| = |i − j|** (same diagonal).
- *Solutions:* 4-Queens has **2 solutions**, (2, 4, 1, 3) and (3, 1, 4, 2). 8-Queens has **92 solutions**. There is **no solution for N = 2 and N = 3**.

Q2. Explain P, NP, NP-Complete and NP-Hard complexity classes and draw the diagram showing how they are related. (7 Marks)

1. Introduction / Definition

Complexity classes group **decision problems** by the computational resources (time) needed to **solve** or **verify** them. **P** is the class of problems **solvable** in polynomial time. **NP** is the class of problems whose solutions can be **verified** in polynomial time. **NP-Complete** problems are the **hardest problems in NP**. **NP-Hard** problems are **at least as hard as every NP problem**, but they need not be in NP.

2. Diagram - Relationship Between the Classes (assuming P ≠ NP)



$$NP\text{-}Complete = NP \cap NP\text{-}Hard; \quad P \subseteq NP$$

Relationship between P, NP, NP-Complete and NP-Hard (assuming P ≠ NP)

3. Core Explanation / Working

(a) Class P (Polynomial time)

- The set of **decision problems solvable by a deterministic algorithm in O(n^k)** time, for some constant k.
- These are considered **"tractable" (efficiently solvable)**.
- *Examples:* sorting O(n log n), binary search O(log n), shortest path, MST, primality testing (AKS).

(b) Class NP (Non-deterministic Polynomial time)

- The set of decision problems whose **"Yes" answer can be verified in polynomial time**, given a **certificate (proposed solution)**.
- Equivalently, the problems solvable in polynomial time by a **non-deterministic** machine (one that "guesses" the right choice).
- **P ⊆ NP**, because if we can solve a problem quickly, we can also verify it quickly.
- *Example:* **Sudoku**. Checking a filled grid is fast, but solving a blank one is hard.

(c) Polynomial-time Reduction (≤p)

- **A ≤p B** means problem A can be **transformed into B in polynomial time**, so that a solution to B gives a solution to A.
- Then **B is at least as hard as A**.

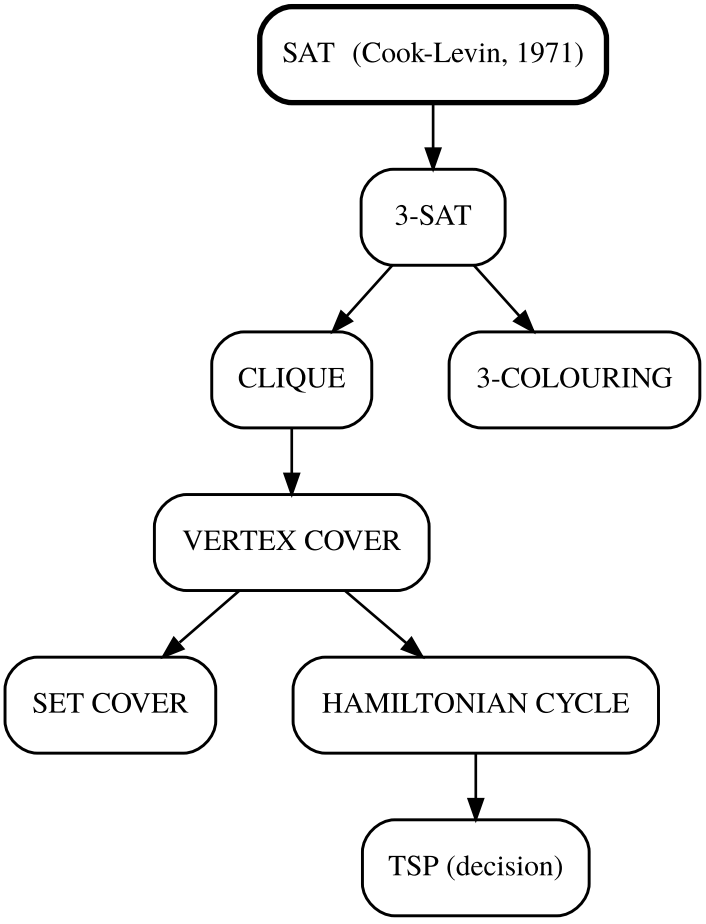
(d) Class NP-Hard

- A problem **H is NP-Hard** if **every problem in NP reduces to H** in polynomial time (L ≤p H for all L ∈ NP).
- It **need not be in NP**: it may not even be a decision problem, and it may be undecidable.
- *Examples:* the **Halting Problem** (undecidable), **TSP optimization**, generalized chess.

(e) Class NP-Complete (NPC)

- A problem **C is NP-Complete** if:
    1. **C ∈ NP** (the solution is verifiable in polynomial time), **and**
    2. **C is NP-Hard** (every NP problem reduces to C).
- So **NPC = NP ∩ NP-Hard**.
- **Cook-Levin Theorem (1971): SAT (Boolean Satisfiability) was the first problem proved NP-Complete.**
- *Proving a new problem X is NPC:* (i) show X ∈ NP, and (ii) reduce a **known NPC problem** to X (for example 3-SAT ≤p Clique ≤p Vertex Cover).
- *Key consequence:* if **any one NPC problem is solved in polynomial time, then P = NP**.

(f) Chain of reductions (standard NPC problems)



Chain of polynomial-time reductions between NP-Complete problems

(g) The P vs NP question

- It is **unknown whether P = NP**. This is one of the seven **Millennium Prize Problems** (with a US $1 million prize).
- Most computer scientists believe **P ≠ NP**.

4. Key Characteristics - Comparison of Classes

Basis	P	NP	NP-Complete	NP-Hard
Solvable in polynomial time	**Yes**	Unknown	Unknown (no known algorithm)	Unknown / maybe never
Verifiable in polynomial time	Yes	**Yes**	**Yes**	Not necessarily
Must be a decision problem	Yes	Yes	Yes	**No** (can be optimization or undecidable)
Belongs to NP	Yes	Yes	**Yes**	Not necessarily
Every NP problem reduces to it	No (unless P = NP)	No	**Yes**	**Yes**
Examples	Sorting, MST, Dijkstra	Sudoku, Factoring, all of P	SAT, 3-SAT, Clique, Vertex Cover	Halting Problem, TSP-optimization

5. Advantages & Disadvantages (of complexity classification)

Advantages

- Tells developers **when to stop looking for an exact efficient algorithm** and use **approximation, heuristics or randomization** instead.
- **Reductions** let one result (for example SAT being NPC) prove the hardness of thousands of problems.
- It is the basis of **modern cryptography**, which relies on some problems being hard.

Disadvantages / Limitations

- **P vs NP is unsolved**, so the classification depends on an unproven assumption.
- Worst-case classification **ignores average-case behaviour**. Many NPC instances are easy in practice (for example SAT solvers).

6. Conclusion / Application

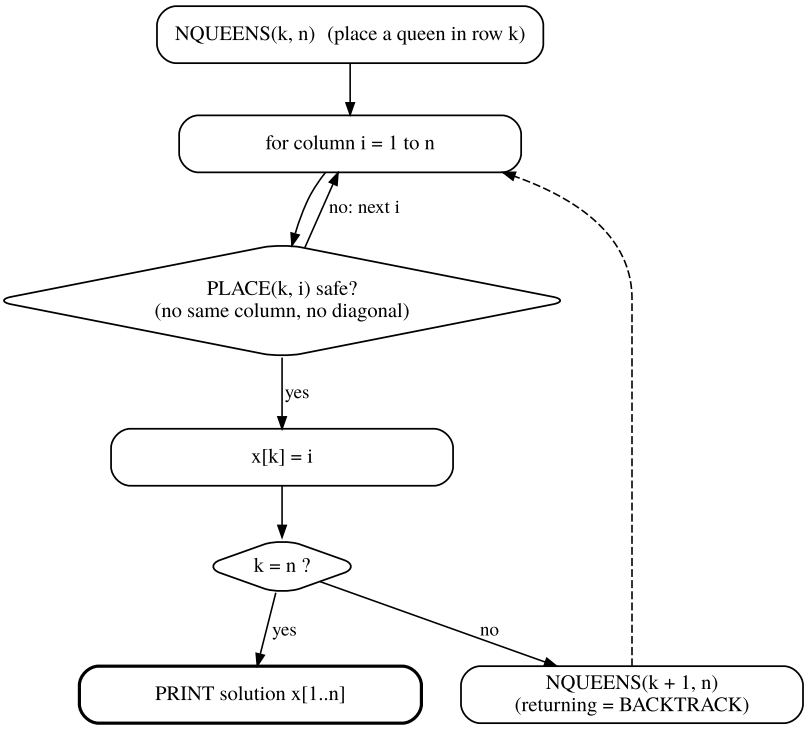
NP-Completeness theory guides real problems like **vehicle routing (TSP), timetable scheduling, VLSI circuit design and RSA cryptography**, where heuristic or approximation algorithms are used.

Q3. Apply the backtracking technique to solve the 4-Queens problem (and its extension to 8-Queens). (7 Marks)

1. Introduction / Definition

The **N-Queens problem** asks us to place N queens on an N × N chessboard so that **no two queens share a row, column or diagonal**. **Backtracking** solves it by placing queens **row by row**. For each row it tries columns 1 to N, keeps a column only if it is **safe**, and **backtracks** to the previous row when no column in the current row is safe.

2. Flowchart and Algorithm



Backtracking algorithm for N-Queens

```
PLACE(k, i)                          // can queen k go in column i?
1. for j = 1 to k - 1
2.     if x[j] == i                          // same column
3.        or |x[j] - i| == |j - k|           // same diagonal
4.         return false
5. return true

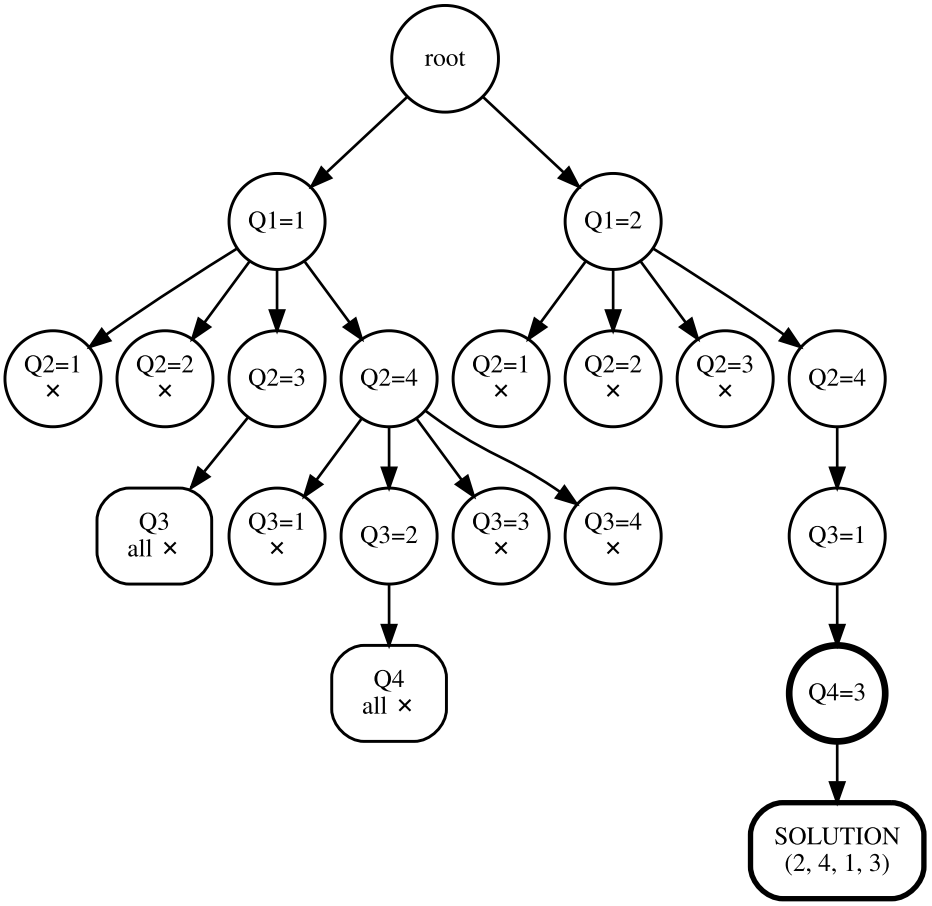
NQUEENS(k, n)
1. for i = 1 to n
2.     if PLACE(k, i)
3.         x[k] = i
4.         if k == n:  print x[1..n]
5.         else:       NQUEENS(k + 1, n)
```

3. Core Explanation / Working - 4-Queens Trace

Step-by-step search (x = (Q1, Q2, Q3, Q4) gives the column of the queen in each row)

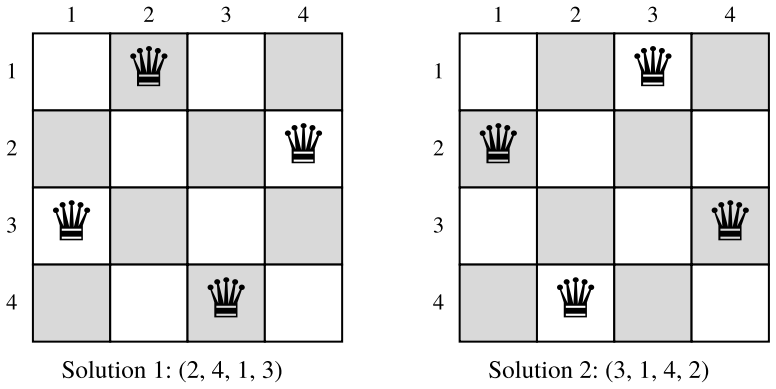
Step	Partial solution	Action	Reason
1	(1)	Q1 at column 1	Empty board, safe
2	(1, 1), (1, 2)	Rejected	Same column / diagonal as Q1
3	(1, 3)	Q2 at column 3	Safe
4	(1, 3, 1..4)	**All rejected**	Col 1, 3 taken; col 2, 4 diagonal with Q2, so **BACKTRACK**
5	(1, 4)	Q2 moved to column 4	Safe
6	(1, 4, 2)	Q3 at column 2	Safe
7	(1, 4, 2, 1..4)	**All rejected**	Col 1, 2, 4 taken; col 3 diagonal with Q3, so **BACKTRACK**
8	(1, 4, 3), (1, 4, 4)	Rejected	Diagonal with Q2 / same column, so **BACKTRACK** to Q1
9	(2)	Q1 moved to column 2	Safe
10	(2, 1..3)	Rejected	Diagonal / column conflict with Q1
11	(2, 4)	Q2 at column 4	Safe
12	(2, 4, 1)	Q3 at column 1	Safe
13	(2, 4, 1, 3)	Q4 at column 3	Safe, **SOLUTION FOUND**

State-space tree (pruned). X = node killed by the bounding function.



Pruned state-space tree for 4-Queens (× = killed by the bounding function)

Solution boards



The two solutions of 4-Queens (Solution 2 is the mirror image of Solution 1)

Extension to 8-Queens

- The same algorithm with n = 8 finds **92 solutions** (12 distinct ones, ignoring rotations and reflections).
- **First solution found: (1, 5, 8, 6, 3, 7, 2, 4).**
- Backtracking explores only a **tiny fraction** of the 8^8 = 16,777,216 possible placements, because invalid partial boards are pruned early.

4. Key Characteristics - Backtracking vs. Brute Force vs. Branch & Bound

Basis	Brute Force	Backtracking	Branch and Bound
Search strategy	Generate all candidates	**DFS with pruning**	BFS / Least-cost search with bounds
Pruning	None	Using a **feasibility (bounding) function**	Using a **cost bound**
Typical use	Small inputs	**Constraint satisfaction** (N-Queens, Sudoku)	**Optimization** (TSP, 0/1 Knapsack)
Nodes explored for 4-Queens	4^4 = 256 boards	**Small pruned tree**	-
Memory	High	**Low** (only the recursion stack)	High (queue of live nodes)

5. Advantages & Disadvantages

Advantages

- **Prunes large parts** of the search space early, which is far faster than brute force.
- **Simple recursive implementation**, needing only O(n) extra space.
- Can find **one solution or all solutions**.

Disadvantages

- The **worst case is still exponential** (O(n!) for N-Queens).
- Performance **depends heavily on the order** in which choices are tried.
- It is impractical for very large N without heuristics.

6. Conclusion / Application

Backtracking, as used for N-Queens, is applied in **Sudoku and crossword solvers, constraint-satisfaction problems (timetabling), VLSI layout and AI game playing**.
